

FactoryBench: Evaluating Industrial Machine Understanding

Anonymous Author(s)

Affiliation

Address

email

Code: anonymous.4open.science/r/FactoryBench

Dataset: huggingface.co/datasets/FactoryBench/FactoryBench

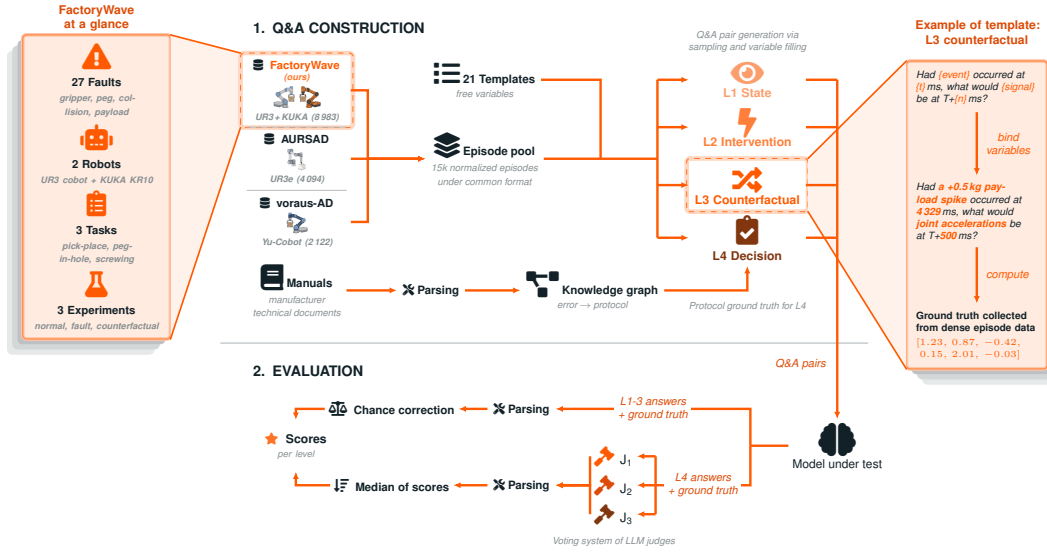


Figure 1: End-to-end FactoryBench pipeline. **(1) Q&A construction:** normalized episodes from FactoryWave, AURSAD [1], and voraus-AD [2] are paired with typed-variable templates (plus the knowledge graph for L4) to generate Q&A items at four reasoning levels. **(2) Evaluation:** the model’s answers are scored deterministically for L1–3 and by the median of three LLM judges for L4 free-form, then chance-corrected per level.

Abstract

Industrial robots emit dense multivariate telemetry that determines their operating state, and acting on a fault requires both reading that state and knowing the procedure that resolves it. These are separate abilities, and they are usually measured separately: perception benchmarks stop at the reading, while control benchmarks score the resulting action without isolating which of the two failed. We introduce **FactoryBench**, a benchmark that measures both on the same episodes from real industrial robots, so that physical perception and physically grounded decision-making can be told apart. Q&A pairs are organized along four levels, physical state estimation, intervention, counterfactual reasoning, and engineering decision-making, and span five answer formats: four are scored deterministically and free-form answers by an LLM-as-judge voting protocol. We release **FactoryWave** (a dense, multitask, multivariate sensor dataset from a UR3 cobot and a KUKA KR10 industrial arm, with counterfactual episodes recorded on hardware by re-executing a task under held-fixed conditions to approximate the do-operation)

and build FactoryBench as over 70k Q&A items grounded in roughly 15k normalized episodes from FactoryWave, AURSAD [1], and voraus-AD [2]. Evaluating six frontier models, none exceeds 50% (chance-corrected) on the perception-side levels, and the ranking reshuffles entirely at the decision level: the leader on the first three levels falls to the bottom of the fourth. Linear probes show this is not one deficit but two, recovering fault presence from a frozen model’s activations at 0.83 where its own answer is below chance (a read-out failure), while fault type is no more decodable than from an untrained network (a representational one). Physical understanding and physically grounded decision-making are dissociable, and current models fail at both in different ways.

1 Introduction

A model can track a physical system accurately and still be useless for acting on it. This gap is easy to state and hard to measure, because the two halves are usually evaluated separately: perception benchmarks score state estimation without asking what follows from it, and control benchmarks score task success without isolating whether failures come from misreading the physics or from mis-planning over a correct reading. For deployment the distinction is the whole question. An operator handed a diagnosis needs to know whether to trust the reading or the recommendation, and a researcher deciding where to spend the next unit of effort needs to know whether the bottleneck is representation or read-out.

We build a benchmark that keeps both halves on the same items so the gap between them is directly observable. Industrial robots are a good place to do this: they are physical systems with contact, payload dynamics, and controller constraints; they emit dense telemetry that fully determines their state; and, unlike most physical-reasoning testbeds, they come with an unambiguous notion of a correct action, since manufacturers publish the recovery procedure for each fault condition. That last property is what makes the decision half gradable rather than a matter of taste. FactoryBench is grounded in **FactoryWave**, a dense multivariate dataset from collaborative and industrial one-arm platforms (UR3 at 125 Hz, KUKA KR10 at 83 Hz) with synchronized setpoint, context, feedback, and effort signals; to our knowledge it is the first extensive anomaly dataset to (partly) cover an industrial robot, a class of machine normally locked behind proprietary controllers and plant confidentiality.

Counterfactual reasoning gets particular attention because it is where physical understanding is usually asserted rather than measured. Simulated environments make counterfactuals free but change the dynamics being tested; real hardware makes them expensive and, strictly, impossible, since two runs never share an identical pre-injection state. We approximate the do-operation empirically instead: for each baseline episode we re-execute the task 3–5 times with every controllable condition held fixed, inject the target fault at a fixed timestep, and retain the run whose pre-injection segment minimizes signature-kernel MMD against the baseline. This yields counterfactual ground truth recorded on the physical system rather than assumed from a simulator, at the cost of being a proxy whose quality we quantify rather than hide. Existing evaluations offer no equivalent: general benchmarks [3, 4, 5] do not probe physical behavior, and time-series Q&A benchmarks are typically synthetic or univariate and omit counterfactuals entirely, while neither specialized time-series models [6, 7, 8] nor LLMs applied to dense numerical series [9, 10] have been tested across perception and decision-making jointly.

Dropping the visual channel is not a difficulty we impose artificially; it is where industry is heading. Plants increasingly run *lights-out* to cut labour and energy costs: FANUC has operated a lights-out robot factory since 2001, producing around 50 robots per day and running unsupervised for up to 30 days, with lighting and HVAC reduced or eliminated because nobody is there who needs them [11]; Xiaomi’s Changping facility builds on the order of 10 million smartphones a year the same way [12]. Most manufacturers adopt this cell by cell rather than plant-wide, but two consequences already follow. The human who would have caught a fault by sight or sound is off the floor, so the diagnostic loop must close automatically. And where cameras are deployed it is overwhelmingly for *product* inspection, pointed at the part rather than the machine [12], while machine health lives in quantities no camera observes: joint torque, motor current, tracking error, controller mode. Controller telemetry is emitted by default by every industrial machine, is the one modality guaranteed across a heterogeneous plant, and is far less restricted than video. We therefore evaluate on telemetry alone.

We evaluate LLMs and LLM-driven agents rather than the specialised classifiers that dominate industrial anomaly detection, because in an unattended plant the artefact that matters is the justification, not the label. A maintenance action (locking out a cell, dismantling a gripper, revising a payload configuration) is itself costly and safety-relevant, so an engineer must audit the reasoning before committing to it. A classifier emits a fault class with no account of which signals produced it; a language model emits a trace from the observed signal to the named root cause to the prescribed procedure, checkable step by step against the telemetry and the vendor manual, and rejectable when the reasoning is unsound even if the label happens to be right. That auditability is what makes a prediction actionable rather than merely accurate, and it is why our decision-making level asks for free-form answers. Downtime sets the stakes: unplanned downtime costs Fortune Global 500 industrial organizations almost \$1.5 trillion a year, 11% of turnover and about \$129 million per facility, and an idle automotive line runs past \$2 million an hour [13].

2 Related work

Recent advances in LLM reasoning include prompting and deliberative strategies (e.g., Chain-of-Thought and Tree-of-Thought) [14, 15], alongside tool-augmented paradigms such as Toolformer and ReAct [16, 17, 18]. Time-series modeling has advanced through transformer-based architectures (Informer, Autoformer, FEDformer, PatchTST) [6, 19, 20, 21], strong alternative baselines such as TFT and N-BEATS [22, 23], and critical comparisons against simpler linear models [24]. Foundation-model directions (e.g., Chronos) explore transfer and zero/few-shot adaptation [25, 26], with large-scale industrial corpora such as FactoryNet [27], on which FactoryBench builds. Anomaly and reliability monitoring literature includes OmniAnomaly, USAD, and Deep SVDD [7, 8, 28], with surveys highlighting persistent gaps in interpretability and root-cause actionability [29, 30].

Causal frameworks provide formal tools for interventions and counterfactuals [31, 32, 33], while temporal methods such as Granger causality and nonlinear causal discovery support directional reasoning in time series [34, 35]. Physical reasoning is most often benchmarked in simulated or rendered environments, where dynamics are synthetic and counterfactual ground truth is generated by re-running the simulator; FactoryBench instead instantiates the association–intervention–counterfactual hierarchy on hardware-recorded telemetry from real robots under closed-loop control, and pairs it with a decision level whose ground truth comes from manufacturer recovery protocols, so physical understanding and its downstream use are measured on the same episodes. Table 1 positions FactoryBench against closely related Q&A and time-series benchmarks along eight axes; Counterfactual denotes Level 3 reasoning over alternative histories (Section 3.1), Free-Form denotes open-ended answers scored by an LLM-as-judge voting protocol, and Novel Dense TS indicates whether the benchmark contributes a new high-frequency multivariate dataset rather than relying exclusively on public sources.

Table 1: Comparison of FactoryBench with existing benchmarks. “Counterfactual” = questions reasoning over alternative histories; “Free-Form” = open-ended answers judged by LLMs; “Novel Dense TS” = contributes a new multivariate high-frequency dataset.

Benchmark	Machine/Robot	Multivariate TS	Number of QAs	Counterfactual	Ranking	Numerical	Free-Form	Novel Dense TS
TimeSeriesExam	×	×	~700	×	✓	×	×	×
ChatTS	×	✓	~134k	×	✓	✓	×	×
EngineMT-QA	✓	✓	~110k	×	✓	✓	✓	×
Time-MQA	×	✓	~200k	×	✓	✓	✓	×
QuAnTS	×	✓	~150k	×	✓	✓	✓	×
CLEVRER	×	×	~305k	✓	×	✓	✓	×
FactoryBench (Ours)	✓	✓	~71k	✓	✓	✓	✓	✓

3 FactoryBench framework

3.1 Four levels of machine understanding

Table 2: FactoryBench levels, what they test, example questions, and expected answer formats.

Level	Type	Example question	What it tests	Answer format
1	State	“We want to isolate the lifting phase in the robot’s time series. Assuming a fixed window length of 15 timesteps, at which timestamp should the window begin?”	Can the model understand what the machine is doing and how it is behaving?	Tensor
2	Intervention	“A collision with a foam cube occurs at $T=850$ ms. Rank signal segments (A–D) in the order you would expect them to appear after the event.”	Can the model reason about how the machine will behave in the face of an event?	Ranking
3	Counterfactual	“Had a payload misconfiguration occurred at $T=200$ ms, what would the target torque on joint 2 at $T+50$ ms have been in this counterfactual case?”	Can the model reason accurately about theoretical scenarios?	Scalar
4	Decision	“Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the root cause and the steps to fix it.”	Can the model make informed decisions about the machine?	Free-form

Our four-tier hierarchy builds on Pearl’s ladder of causation association, intervention, and counterfactual reasoning, the organizing principle for causal inference and an increasingly common evaluation axis for machine-learning systems [31, 32, 41, 42, 43]. We add a fourth decision-making level reflecting how industrial robotic platforms are actually operated: after interpreting state and causal relationships, operators must select and execute corrective procedures, typically prescribed by vendor manuals. This aligns with the diagnose-then-act loop standard in fault-tolerant control, and grounds each level in a qualitatively distinct reasoning skill with interpretable failure modes.

Level 1: State. Assesses the agent’s ability to interpret the current state of the machine under normal conditions, including identification of operational modes and recognition of sensor patterns.

Level 2: Intervention. The agent must reason about the consequences of interventions or events occurring at the present timestep, including predicting the immediate impact of control actions and diagnosing emerging faults in real time.

Level 3: Counterfactual. Evaluates reasoning about hypothetical scenarios, requiring the agent to simulate alternative histories and assess how outcomes would differ under counterfactual conditions.

Level 4: Decision Making. The highest level evaluates the agent’s ability to act on its understanding of the system: given a recorded episode and a task description, the agent must produce a diagnosis or remediation grounded in the previous three levels, reflecting what we expect from an LLM acting as an engineering assistant on the factory floor.

Each question is emitted in one of five answer formats (single-select MCQ, multi-select MCQ, ranking, tensor, free-form); the four structured formats are scored deterministically and free-form answers are scored by an LLM-as-judge voting protocol.

3.2 Time series data and causal schema (SCE)

So that the dataset structure itself encodes causality, all time-series signals are organized into three causal groups: **Setpoint** (the controller’s commanded target), **Context** (physical and configured conditions), and **Effort+Feedback** (the machine’s measured response). Under healthy operation the response is a known function of setpoint and context; faults manifest as structured residuals from that baseline, giving a single operational fault definition that applies across machines and tasks.

The data conforms to this schema across two source types: **FactoryWave**, a custom dataset generated from one-arm robotic platforms executing industrial tasks under varied conditions and systematically injected anomalies, and **open-source datasets** such as AURSAD [1] and voraus-AD [2], preprocessed

138 to conform to the unified episode structure. All three sources provide the full setpoint–context–effort
 139 triplet at 83 Hz or higher (Table 3).

Table 3: Datasets incorporated into FactoryBench. FactoryWave is collected and released with this work; the other two are open-source datasets adapted to our schema. Tasks: PnP = pick-and-place, Scr = screwing, PiH = peg-in-hole.

Dataset	Robot	Episodes	Frequency	Tasks	Anomalies
AURSAD [1]	UR3e	4094	100 Hz	Scr	4
voraus-AD [2]	Yu-Cobot	2122	100/500 Hz	PnP	12
FactoryWave (ours)	UR3, KUKA KR10	8983	125/83 Hz	PnP, Scr, PiH	27

140 4 Reliable Q&A generation

141 4.1 Scalable generation via extensive labeling

142 Scalable ground-truth generation is the central challenge of any Q&A benchmark grounded in raw
 143 data. FactoryBench couples a structured labeling ontology with a context-free-grammar-style template
 144 system: each of the 21 templates contains variable slots filled at generation time either from label
 145 information read directly from the episode (signal names, timestamps, anomaly labels, root causes)
 146 or from a predefined sampling distribution attached to that slot (prediction horizons, thresholds,
 147 curated vocabularies). Answer options for single- and multi-select questions are drawn from a shared
 148 pool of pre-defined verifiable statements, each paired with a rule evaluated deterministically against
 149 the time series, so ground truth is never imputed but computed directly from the underlying labels;
 150 for single-select MCQ, the sampling step enforces that exactly one of the four options evaluates
 151 to true, so the question is unambiguous by construction. This is also our primary defense against
 152 template-generated benchmarks being solvable from surface structure alone: every template admits a
 153 combinatorially large instantiation space over signal names, timestamps, prediction horizons, and
 154 injected-fault types, so the correct answer is never recoverable from the question text alone and is
 155 always determined by the paired time series, which varies across instantiations. Alongside the full
 156 corpus we release **FactoryBench-Lite**, a 3,000-item subset balanced across all 21 templates and,
 157 within each template, on the dimension that determines its answer, so that a cheap evaluation is not
 158 weighted by how many items a template happens to contribute.

159 4.2 Density of FactoryWave

160 FactoryWave is collected from two physical robots (UR3 and the KUKA KR10; Figure 2) executing
 161 up to three industrial tasks each: pick-and-place, screwing, and peg-in-hole. Each complete task
 162 cycle constitutes one episode, recorded at 125 and 83 Hz across over 100 sensor channels covering
 163 joint setpoints, feedback, torques, speeds, estimated contact forces, TCP pose, gripper state, and
 164 task-phase labels. To compensate for the limited tool-side telemetry exposed by the KUKA KSS
 165 controller, we additionally mount a 9-DoF inertial measurement unit on the KUKA gripper, streamed
 166 in sync with the controller signals.

167 Episodes are organized into three experiment types. **Normal** episodes capture nominal operation
 168 across three payload levels. **Fault** episodes inject one of 27 fault types (gripper failures, miscon-
 169 figurations, collisions, and peg-in-hole-specific anomalies), each with fault-specific metadata and,
 170 where applicable, a precise injection timestep. **Counterfactual** episodes provide ground truth for
 171 Level 3 reasoning by approximating the do-operation on physical hardware: for each baseline we
 172 re-execute the task 3–5 times with every controllable condition held fixed and inject the target fault
 173 at a fixed timestep t , then keep the run whose pre-injection segment minimizes the signature kernel
 174 MMD against the baseline, using it as an approximate ground truth for hypothetical divergence.

175 4.3 Knowledge graph and protocol extraction

176 Reliable ground truth for Level 4 troubleshooting requires machine-specific recovery protocols
 177 grounded in manufacturer documentation. For each robot we parse the manufacturer’s official runtime
 178 error documentation into a structured mapping from error codes to descriptions and recommended



Figure 2: FactoryWave tasks and fault catalogue. Photographs of the robotic platforms (UR3, KUKA KR10) executing the three tasks (pick-and-place, peg-in-hole, screwdriving) and a representative subset of the 27 systematically injected fault conditions.

recovery steps, then map every anomaly type in FactoryBench to the most likely corresponding manufacturer error code, with PhD-level robotics experts reasoning about the physical cause of each anomaly. Where a physical fault injection has no well-defined controller error, the same experts author the recovery protocol directly. The complete mapping is ingested into a knowledge graph alongside robot specifications, task definitions, and signal schema, which the generation pipeline queries at question-generation time to inject ground-truth protocol context into Level 4 items without per-question human review.

5 Evaluation and results

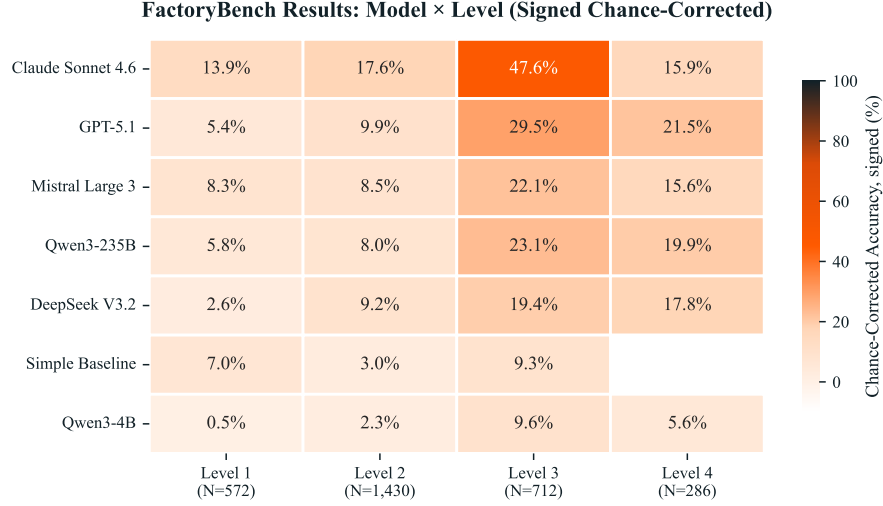
5.1 Zero-shot evaluation

We evaluate a cross-vendor panel of frontier LLMs available at submission time (April 2026): Claude Sonnet 4.6 [44] and GPT-5.1 [45] on the closed-weight side, and DeepSeek V3.2 [46], Mistral Large 3 [47], and Qwen3-235B [48] on the open-weight side, plus Qwen3-4B [48] as a lightweight reference. For calibration we report a non-LLM baseline that dispatches per item by answer format: linear regression on the target signal for scalar and tensor templates, and uniform random over the option set for single- and multi-select MCQ and ranking templates (free-form Level 4 templates are excluded, as no traditional method maps cleanly to producing a remediation protocol). Figure 3 reports chance-corrected accuracy for the full panel.

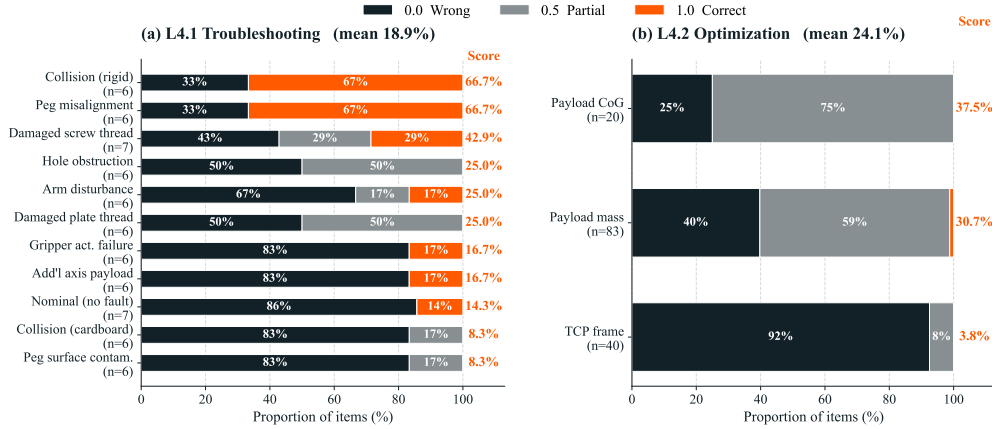
5.2 Signal comprehension across Levels 1–3

Most models fail to separate from the simple baseline on Level 1, including the largest in the panel. Under the signed chance correction the non-LLM baseline lands at +4.3% on L1. Four of six LLMs fail to clear it: Qwen3-4B (−1.8%) and DeepSeek V3.2 (−0.3%) score below chance outright, and GPT-5.1 (+3.2%) and Qwen3-235B (+3.6%) sit below the baseline despite being the largest models in the panel. Only Mistral Large 3 (+5.9%) and Claude Sonnet 4.6 (+11.8%) clear it, and Mistral only barely. Raw scale does not reliably confer the ability to extract precise state from dense industrial signals.

Models that struggle on Level 1 still outperform the baseline on Levels 2 and 3, and rank order is stable across the three. DeepSeek V3.2 (+5.8%, +17.1%) and GPT-5.1 (+6.9%, +27.5%) both lift clear of the L2/L3 baseline values (−0.2%, +6.2%) despite trailing it on L1, yet the model rank order is nearly identical across L1–L3: Claude Sonnet 4.6 leads at L2 and L3 (+14.6%,



(a) Chance-corrected accuracy (%) on FactoryBench’s four reasoning levels.



(b) GPT-5.1 L4 score distribution by ground-truth root cause (left, troubleshooting) and misconfigured parameter (right, optimization). Each bar shows the proportion of items scored 0/0.5/1; the right-edge value is the per-category mean accuracy.

Figure 3: Main results. (a) Cross-model panel performance across all four reasoning levels. (b) Per-fault and per-parameter breakdown of the GPT-5.1 L4 mass.

+45.7%), Qwen3-235B is the strongest open-weight model (+20.6% on L3), and Mistral Large 3 follows (+5.3%, +19.7%). This points toward state comprehension propagating into intervention and counterfactual reasoning rather than being substituted by it.

A human expert reaches near-ceiling on Levels 1–2, but not on Level 3. A robotics expert answered a 60-item sample under open-tool, no-LLM conditions, graded by the benchmark’s own scorer. On raw scores over the identical items the expert reaches 0.962 at L1 and 0.980 at L2, against 0.712 and 0.662 for the strongest model and 0.35–0.52 for the rest: the gap is not an artifact of ambiguous items, the information is in the released signals, and reading it out is what the panel fails at. At L3 the expert reaches only 0.625 and does not lead the panel, which we attribute to counterfactual prediction being poorly served by existing time-series tooling rather than to defective items.

5.3 Engineering decision-making at Level 4

Level 4 exposes an industrial readiness gap, and the panel ordering reshuffles completely. On Level 4 (uncorrected 0/0.5/1 rubric, not directly comparable to the chance-corrected L1–L3 scores)

the leader reaches 21.5% and the other five score between 5.6% and 19.9%. Claude Sonnet 4.6, which leads L1–L3 and more than doubles the next model on L3, falls to fourth of six on L4 at 15.9%. GPT-5.1 leads L4 at 21.5%, and Qwen3-235B (19.9%) and DeepSeek V3.2 (17.8%) also overtake Claude Sonnet 4.6 there. The dissociation between *signal comprehension* and *protocol retrieval* means optimizing for one does not deliver the other: even models with strong L1–L3 comprehension either lack the machine-specific knowledge to diagnose faults, or cannot ground that knowledge in the observed signals, and closing this gap is the central challenge FactoryBench is designed to measure.

The L4 collapse is partly a read-out failure, not only a representational one. We probe this directly: linear probes on a frozen open-weight panel member recover fault *presence* from the model’s activations well above chance (0.83) even where the model’s own answer stays below chance, whereas fault *type* is only weakly decodable and no better than an untrained-network baseline. The information a correct L4 answer needs is thus partly already in the model’s representations; part of what fails is turning it into a verbalized diagnosis, not only building it in the first place.

GPT-5.1’s L4 failures are asymmetric between troubleshooting and optimization. On troubleshooting ($n=143$, mean 0.189; Figure 3b, left) the score distribution is sharply bimodal: 76.2% zero, 14.0% perfect, only 9.8% partial. Because the evaluation subset draws root causes evenly, the perfects separate by fault physics rather than by class frequency: peg-insertion misalignment and rigid-object collision reach 0.67, while every fault that changes assembly state without a mechanical transient (missing screw, extra component, loosening phase, unstable mounting) scores exactly 0.00. On optimization ($n=143$, mean 0.241; Figure 3b, right) GPT-5.1 scores perfectly exactly once and is the only model in the panel to do so at all: ground-truth answers are single-parameter corrections, but it emits lists of generic motion-tuning suggestions and treats payload configuration as one undifferentiated item among many, earning 0.5 at best. Even where GPT-5.1 leads L4, it remains weak at decision-making in an industrial context: it reaches 0.31 on payload mass but only 0.04 on TCP frame misconfiguration.

5.4 Limitations

FactoryBench has three substantive limitations. Level 3 *counterfactual* ground truth on physical hardware is fundamentally approximate: two real runs cannot share an identical pre-injection state, so our signature-kernel-MMD-selected pair is the closest empirical proxy for the do-operation rather than the do-operation itself, and L3 scores conflate model reasoning with proxy quality. As the first benchmark of its sort our fault model is simplified: industrial faults in production are typically compound and gradual, while ours are atomic, fast, and drawn from a closed catalogue of 27 injected mechanisms, making FactoryBench closer to in-distribution fault *recognition* than the fault *detection* problem operators actually face. Level 4 ground truth is drawn from a finite catalogue of manufacturer error codes and expert-authored recovery procedures, so a model is effectively evaluated on *closed-book retrieval* of protocols it may or may not have seen during pretraining rather than on open-ended engineering reasoning, conflating “does the model know this vendor manual?” with “can it reason from signals to a corrective action?”.

6 Conclusion

We introduced FactoryBench, a four-level benchmark for machine understanding over industrial robotic telemetry, grounded in FactoryWave and structured around Pearl’s causal hierarchy. Zero-shot evaluation of six frontier LLMs shows that signal comprehension and protocol-grounded decision-making are distinct capabilities that do not co-develop: the panel ordering on Levels 1–3 reshuffles on Level 4, with GPT-5.1 taking the L4 lead despite being mediocre on the structured levels while the L1–L3 leader ranks near the bottom on L4. Structured-level scores remain well below 50% (chance-corrected) and no model approaches saturation on any level. For physical understanding aimed at decision-making, the practical lesson is that the two must be measured jointly: our panel’s perception-side ranking does not predict its decision-side ranking, and the probing analysis shows the decision failures decompose into a calibration problem where the physical concept is present and a representation problem where it is not, which are not addressed by the same interventions. Full dataset details, extended per-format and per-robot breakdowns, the human-baseline protocol, and the

full probing methodology are available in the extended paper and released alongside the dataset and code (links above).

References

- [1] Błażej Leporowski, Daniella Tola, Casper Hansen, and Alexandros Iosifidis. AURSAD: Universal robot screwdriving anomaly detection dataset. *arXiv preprint arXiv:2102.01409*, 2021.
- [2] Jan Thieß Brockmann, Marco Rudolph, Bodo Rosenhahn, and Bastian Wandt. The voraus-AD dataset for anomaly detection in robot applications. *IEEE Transactions on Robotics*, 40:438–451, 2024. arXiv:2311.04765.
- [3] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- [4] Aarohi Srivastava et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023.
- [5] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. MMMU: A massive multi-discipline multi-modal understanding and reasoning benchmark for expert AGI. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [6] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 11106–11115, 2021.
- [7] Ya Su, Youjian Zhao, Chenhao Niu, Rong Liu, Wei Sun, and Dan Pei. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- [8] Julien Audibert, Pietro Michiardi, Frédéric Guyard, Sébastien Marti, and Maria A. Zuluaga. USAD: Unsupervised anomaly detection on multivariate time series. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2020.
- [9] Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. Large language models are zero-shot time series forecasters. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y. Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. Time-LLM: Time series forecasting by reprogramming large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [11] International Manufacturing Technology Show (IMTS). Automated factory guide: Lights-out and dark manufacturing. <https://www.imts.com/read/article-details/Automated-Factory-Guide-Lights-Out-and-Dark-Manufacturing/1206/type/Read/1>. Accessed 2026-08-10.
- [12] Metrology and Quality News. Fully autonomous ‘dark’ smart factory runs 24/7 without human intervention. <https://metrology.news/autonomous-dark-smart-factory-runs-24-7-without-human-intervention/>, 2024. Accessed 2026-08-10.
- [13] Senseye Predictive Maintenance, Siemens. The true cost of downtime 2022. Technical report, Siemens, 2023. <https://assets.new.siemens.com/siemens/assets/api/uuid:3d606495-dbe0-43e4-80b1-d04e27ada920/dics-b10153-00-7600truecostofdowntime2022-144.pdf>.
- [14] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

- [15] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [16] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [17] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [18] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zhiyuan Zeng, Yujia Huang, Chaojun Xiao, Chi Han, et al. Tool learning with foundation models. *ACM Computing Surveys*, 2024. arXiv:2304.08354.
- [19] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [20] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International conference on machine learning*, pages 27268–27286. PMLR, 2022.
- [21] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers. arxiv 2022. *arXiv preprint arXiv:2211.14730*, 2022.
- [22] Bryan Lim, Sercan Ö. Arik, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.
- [23] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.
- [24] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. Are transformers effective for time series forecasting? In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [25] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024. arXiv:2403.07815.
- [26] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024. arXiv:2310.10688.
- [27] Karim Othman, Jonas Petersen, Matei Ignuta-Ciuncanu, Camilla Mazzoleni, Federico Martelli, Alessandro Lombardi, Riccardo Maggioni, and Philipp Petersen. Factorynet: A large-scale dataset toward industrial time-series foundation models, 2026.
- [28] Lukas Ruff, Robert A. Vandermeulen, Nico Görnitz, Lucas Deekce, Shoaib Ahmed Siddiqui, Alexander Binder, Emmanuel Müller, and Marius Kloft. Deep one-class classification. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2018.
- [29] Raghavendra Chalapathy and Sanjay Chawla. Deep learning for anomaly detection: A survey. *arXiv preprint arXiv:1901.03407*, 2019.

- [30] Alexander Lavin and Subutai Ahmad. Evaluating real-time anomaly detection algorithms: The Numenta anomaly benchmark. In *Proceedings of the IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 38–44, 2015.
- [31] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.
- [32] Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. *Elements of Causal Inference*. MIT Press, 2017.
- [33] Donald B. Rubin. Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology*, 66(5):688–701, 1974.
- [34] Clive W. J. Granger. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 37(3):424–438, 1969.
- [35] Jakob Runge, Peer Nowack, Marlene Kretschmer, Seth Flaxman, and Dino Sejdinovic. Detecting and quantifying causal associations in large nonlinear time series datasets. *Science Advances*, 5(11):eaau4996, 2019.
- [36] Yifu Cai, Arjun Choudhry, Mononito Goswami, and Artur Dubrawski. TimeSeriesExam: A time series understanding exam, 2024.
- [37] Zhe Xie, Zeyan Li, Xiao He, Longlong Xu, Xidao Wen, Tieying Zhang, Jianjun Chen, Rui Shi, and Dan Pei. ChatTS: Aligning time series with LLMs via synthetic data for enhanced understanding and reasoning. *Proceedings of the VLDB Endowment*, 18(8):2385–2398, 2025.
- [38] Yilin Wang, Peixuan Lei, Jie Song, Yuzhe Hao, Tao Chen, Yuxuan Zhang, Lei Jia, Yuanxiang Li, and Zhongyu Wei. ITFormer: Bridging time series and natural language for multi-modal QA with large-scale multitask dataset, 2025.
- [39] Yaxuan Kong, Yiyuan Yang, Yoontae Hwang, Wenjie Du, Stefan Zohren, Zhangyang Wang, Ming Jin, and Qingsong Wen. Time-MQA: Time series multi-task question answering with context enhancement, 2025.
- [40] Felix Wenkel, Ghada Sokar, Yuan Zhang, and Mirco Ravanelli. QuAnTS: Question answering on time series. *arXiv preprint arXiv:2411.04795*, 2024.
- [41] Zhijing Jin, Yuen Chen, Felix Leeb, Luigi Gresele, Ojasv Kamal, Zhiheng Lyu, Kevin Blin, Fernando Gonzalez Adauto, Max Kleiman-Weiner, Mrinmaya Sachan, and Bernhard Schölkopf. CLadder: A benchmark to assess causal reasoning capabilities of language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [42] Nick Pawlowski, Daniel C. Castro, and Ben Glocker. Deep structural causal models for tractable counterfactual inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [43] Maxime Peyrard and Robert West. A ladder of causal distances. *arXiv preprint arXiv:2005.02480*, 2020.
- [44] Anthropic. Claude Sonnet 4.6 system card, 2026. Anthropic system card, February 2026. <https://www.anthropic.com/claude-sonnet-4-6-system-card>.
- [45] OpenAI. GPT-5.1 system card, 2025. OpenAI system card addendum. <https://openai.com/index/gpt-5-system-card-addendum-gpt-5-1/>.
- [46] DeepSeek-AI. DeepSeek-V3.2: Pushing the frontier of open large language models, 2025. arXiv:2512.02556. <https://arxiv.org/abs/2512.02556>.
- [47] Mistral AI. Mistral Large 3: Model card, 2025. Mistral AI announcement. <https://mistral.ai/news/mistral-3>.
- [48] Qwen Team. Qwen3 technical report, 2025. arXiv:2505.09388.

- 413 [49] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier
414 probes. *arXiv preprint arXiv:1610.01644*, 2016.
- 415 [50] Ömer Şahin Taş and Royden Wagner. Words in motion: Extracting interpretable control vectors
416 for motion transformers. In *International Conference on Learning Representations (ICLR)*,
417 2025. arXiv:2406.11624.
- 418 [51] John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In
419 *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*
420 *(EMNLP)*, 2019.

A Answer formats and scoring

FactoryBench emits questions in five answer formats. The four structured formats (single-select MCQ, multi-select MCQ, ranking, tensor) are scored deterministically by per-format rules; free-form answers, used only at Level 4, are scored by an LLM-as-judge voting protocol whose inter-judge reliability we analyse in Appendix A.2. Each format is designed to remain hard to answer in the absence of correct reasoning over the time series. Table 4 summarises the raw scoring rules; the rest of this section gives the implementation details together with the chance-correction step that places every format on a comparable scale before aggregation.

Table 4: Answer formats used in FactoryBench and their raw scoring rules. The chance-correction step described in this section is applied on top of these rules before per-level aggregation. Free-form answers (Level 4 only) are scored by an LLM-as-judge voting protocol with three judges (GPT-5.1, Claude Sonnet 4.6, DeepSeek V3.2) aggregated by the median of the three votes.

Format	Description	Raw scoring (before chance correction)
Single-select MCQ	Three or four mutually exclusive options (A–C or A–D); the model selects one.	Exact match: 1 if the parsed letter equals the gold letter, 0 otherwise. Per-item chance level $E = 1/k$ where $k \in \{3, 4\}$ is the option count for that question.
Multi-select MCQ	Four independent statements (A–D) about the outcome of an event or intervention; the model emits a T/F string of length four.	Positional fraction: $1/4$ per slot whose T/F matches the gold; raw score in $\{0, 0.25, 0.5, 0.75, 1\}$. Chance level $E = 1/2$.
Ranking	Four time-series segments or outcomes (A–D) ordered by a specified criterion. Answer is a permutation string (e.g. DABC).	Positional fraction: $1/4$ per slot whose letter matches the gold permutation; raw score in $\{0, 0.25, 0.5, 0.75, 1\}$. Chance level $E = 1/4$.
Tensor	One or more scalar values (e.g. expected sensor reading after an intervention). Answer is a list of floats.	Per-scalar three-level piecewise score: 1 if $ \hat{v} - v^* \leq m$, 0.5 if $m < \hat{v} - v^* \leq 2m$, else 0. Per-channel margin calibrated to $m_j = R_j/12$ so the chance level matches MCQ ($E = 1/4$).
Free-form	Open-ended natural-language response, used only for the two Level 4 question types: troubleshooting and optimization.	Score $\in \{0, 0.5, 1\}$ by LLM-as-judge against the reference answer: 1.0 if semantically equivalent to the reference, 0.5 if partially correct or imprecise but on-topic, 0.0 if incorrect, irrelevant, or refusing. The verbatim judge prompt is given in Appendix A.

Chance correction. Without correction, formats with different chance baselines are not directly comparable: a model scoring 0.5 on a multi-select item (chance $E = 1/2$) has demonstrated nothing, while a model scoring 0.5 on a single-select item (chance $E = 1/4$) has demonstrated above-chance performance. We therefore transform every raw item score s into a chance-corrected score

$$\tilde{s} = \frac{s - E}{1 - E},$$

where E is the format’s chance level. By linearity of expectation, uniform random guessing produces $\mathbb{E}[\tilde{s}] = 0$ for every format by construction; $\tilde{s} = 1$ corresponds to a perfect answer. The transform is signed: items scoring below chance receive $\tilde{s} < 0$, so a model that is worse than uniform random is visibly penalised in the aggregate mean rather than hidden by a floor. The per-format floor is $\tilde{s}_{\min} = -E/(1 - E)$, which ranges from -1 for multi-select ($E = 1/2$) to $-1/3$ for tensor / ranking / four-way MCQ ($E = 1/4$); aggregate means below zero should therefore be interpreted with the level’s format mix in mind. The non-LLM baseline used in Section 5.1 doubles as the empirical audit: after chance correction it hovers near 0 on every format (+1.9%, -3.0%, +1.7% on Levels 1–3 respectively), which we verify before treating any model score as evidence of reasoning.

Single-select MCQ. Raw score $s \in \{0, 1\}$. The chance level is read off the question payload as $E = 1/k$. The corrected score is $\tilde{s} = (ks - 1)/(k - 1)$, mapping a correct answer to $\tilde{s} = 1$ and a wrong answer to $\tilde{s} = -1/(k - 1)$ (i.e. $-1/2$ for $k = 3$, $-1/3$ for $k = 4$).

445 **Multi-select MCQ.** Raw score $s \in \{0, 0.25, 0.5, 0.75, 1\}$ with chance level $E = 1/2$ (each T/F
 446 slot matches the gold with probability $1/2$ under uniform random guessing). The corrected score is
 447 $\tilde{s} = 2s - 1$, mapping $\{0, 0.25, 0.5, 0.75, 1\}$ to $\{-1, -0.5, 0, 0.5, 1\}$.

448 **Ranking.** Raw score is the positional-match fraction over a permutation of length four. The
 449 expected number of fixed points of a uniformly random permutation is exactly 1 regardless of
 450 length, so the chance level is $E = 1/n = 1/4$. The corrected score is $\tilde{s} = (4s - 1)/3$, mapping
 451 $\{0, 0.25, 0.5, 0.75, 1\}$ to $\{-1/3, 0, 1/3, 2/3, 1\}$.

452 **Tensor.** Each scalar j in a vector answer is scored on a three-level scale: 1 if $|\hat{v}_j - v_j^*| \leq m_j$,
 453 0.5 if within $2m_j$, 0 otherwise. The raw item score is the average over the n scalars, $s = \frac{1}{n} \sum_j s_j$.
 454 The discrete form mirrors MCQ/ranking and still credits near-misses inside $2m_j$. The per-channel
 455 margin is calibrated to $m_j = R_j/12$, where R_j is the channel’s empirical $[p_2, p_{98}]$ range across the
 456 unified episodes (precomputed once); this sets the piecewise scorer’s chance expectation to $E = 1/4$,
 457 matching single-select MCQ. The corrected item score follows the MCQ form, $\tilde{s} = (4s - 1)/3$.
 458 Scalars with a degenerate channel range ($R_j \approx 0$) are dropped from the average.

459 **Free-form.** Free-form answers do not admit a meaningful chance baseline: the answer space is
 460 unbounded natural language, so there is no uniform random distribution to subtract. We therefore
 461 leave free-form scores uncorrected. The lack of correction does not introduce cross-format bias
 462 because free-form is used *only* at Level 4, and Level 4 contains no other answer format, so free-form
 463 scores are never aggregated alongside structured-format scores at the same level. As a consequence,
 464 Level 4 scores and Levels 1–3 scores are reported on different underlying scales (an uncorrected
 465 0/0.5/1 rubric versus a chance-corrected structured metric) and are *not directly comparable in*
 466 *magnitude*; when we contrast them in Section 5.1 we refer to rank-order and gap-to-baseline effects
 467 rather than absolute-difference claims.

468 Every free-form answer is scored by the LLM judge against the gold reference answer (assembled
 469 from the FactoryBench knowledge graph) using the prompt below, reproduced *verbatim* from the
 470 scoring code and applied identically to both Level 4 question types (troubleshooting and optimization).
 471 The placeholders {question}, {reference}, and {prediction} are filled per item:

```

472 You are an expert evaluator for a robotics sensor-data Q&A benchmark.
473 Score a model’s free-form answer against a reference answer.
474
475 Scoring:
476   1.0 (Correct):  semantically equivalent to the reference: it gives the correct
477                   root cause / misconfigured parameter AND a plausible corrective
478                   step (exact wording of the fix is NOT required).
479   0.5 (Neutral):  on-topic AND it identifies the correct root cause / misconfigured
480                   parameter, even if imprecise or without the corrective step.
481   0.0 (Wrong):    incorrect, irrelevant, names the WRONG root cause, is a generic
482                   list that MISSES the specific parameter, claims a fault when the
483                   reference says the machine is normal (or vice versa), or refuses
484                   to answer.
485
486 Being merely "on-topic" is NOT enough for 0.5: the answer must identify the actual
487 root cause / parameter from the reference. A generic list of tuning suggestions
488 that does not single out the reference’s specific cause/parameter is 0.0.
489
490 Respond with ONLY a JSON object on a single line, no markdown:
491 {"score": <0.0|0.5|1.0>, "reason": "<one short sentence>"}
492
493 Question:
494 {question}
495
496 Reference answer:
497 {reference}
498
499 Model answer:
500 {prediction}

```

501 A held-out audit confirms the panel applies this rubric consistently: an independent grader (outside
502 the three-judge panel) given this exact prompt matches the judge-median on 95% of a representative
503 100-item sample (quadratic-weighted $\kappa = 0.79$).

504 **Aggregation.** Per-question chance-corrected scores $\tilde{s}$ are averaged uniformly within a template and
505 level.

506 A.1 A caveat on the tensor chance level at L3.5

507 The chance level for the tensor and numerical formats is not measured but derived: the three-level
508 piecewise scorer with per-channel margin $m_j = R_j/12$ is constructed to give $E = 3m/R = 1/4$
509 under uniform guessing. That derivation assumes the guess is uniform over the channel’s range.
510 It is a poor description of a forecasting template, where the quantity being predicted is strongly
511 autocorrelated and a constant-continuation guess is far better than uniform.

512 The non-LLM baseline makes the gap visible. On L3.5 (trajectory prediction) it scores +37.2%
513 chance-corrected, which is not a possible outcome for a method with no access to the answer if
514 $E = 1/4$ were the operative chance level for that template; the same baseline scores −17.8% on L3.4
515 and −0.2% on the L2 aggregate. The natural reading is that $m_j = R_j/12$ is too generous for L3.5
516 specifically, so a prediction that simply continues the recent trajectory lands inside the margin more
517 often than the calibration assumes.

518 Two consequences. First, every model’s L3 aggregate is inflated by whatever share L3.5 contributes,
519 and cross-level comparisons involving L3 should be read with that in mind; the ordering *within* L3 is
520 unaffected, since all models face the same margin. Second, L3.4 is not implicated: the baseline is
521 well below chance there while models reach +19.7% to +63.4%, so that template’s separation is real.
522 We report the numbers as measured rather than recalibrating m_j post hoc, since changing the margin
523 would silently redefine the metric and break comparability with previously published FactoryBench
524 results; the baseline row is included in Figure 3 precisely so a reader can see the size of the effect.

525 A.2 Inter-judge reliability of the Level-4 panel

526 Because free-form Level-4 answers are scored by three judges (GPT-5.1, Claude Sonnet 4.6, DeepSeek
527 V3.2) aggregated by median (Table 4), the L4 scores inherit whatever (dis)agreement those judges
528 carry. We therefore quantify their reliability. We pool every Level-4 answer for which all three judges
529 returned a valid verdict across the six-model evaluatee panel ($n=6,617$ answers, three verdicts each
530 in $\{0, 0.5, 1\}$); inter-rater agreement is a property of the judges, so pooling across evaluatees is the
531 appropriate unit of analysis.

532 **The panel is reliable.** Agreement is substantial-to-good: Fleiss’ $\kappa = 0.60$ and interval Krippen-
533 dorff’s $\alpha = 0.78$ across the three raters, with 87.5% of items unanimous, 11.9% carrying a 2-of-3
534 majority, and only 0.6% three-way splits (Figure 4b). Pairwise, exact agreement is 89.7–93.4%
535 (mean 91.5%) and the quadratic-weighted κ is 0.74–0.81 (Figure 4a), with GPT-5.1 and DeepSeek
536 agreeing most and Claude and DeepSeek least. The pairwise confusion matrices are strongly diagonal
537 and disagreements are almost never polar ($0 \leftrightarrow 1$): they are overwhelmingly between adjacent grades
538 ($0 \leftrightarrow 0.5$ or $0.5 \leftrightarrow 1$), consistent with the ordinal rubric.

539 **A small leniency spread justifies median aggregation.** The judges differ only mildly in strictness:
540 mean verdict 0.101 (Claude), 0.082 (GPT-5.1), 0.066 (DeepSeek), so Claude is the most lenient and
541 DeepSeek the strictest. This directional bias is visible in the off-diagonals of the confusion matrices
542 (e.g. Claude assigns 0.5 where DeepSeek assigns 0 on 8% of items), and it is exactly the regime
543 in which the *median* is preferable to the mean: it is robust to a single lenient or strict judge and
544 returns the majority verdict whenever one exists, which it does for 99.4% of items. High agreement
545 together with the robustness of median aggregation indicates that the reported Level-4 scores are not
546 an artefact of any single judge.

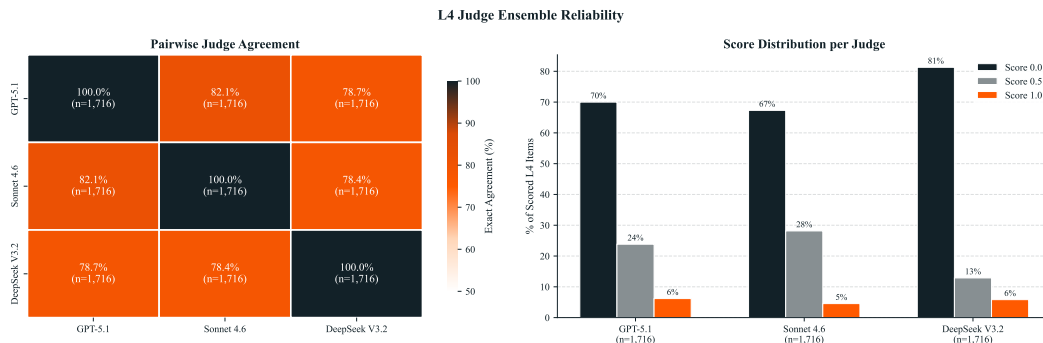


Figure 4: **The three-judge Level-4 panel is reliable.** (a) Pairwise exact agreement (91.3%, 93.4%, 89.7%; mean 91.5%) and quadratic-weighted κ for each judge pair, with the 3-way Fleiss $\kappa=0.60$ and Krippendorff $\alpha=0.78$ as reference. (b) Consensus across the three judges: all three agree on 87.5% of items, 11.9% are 2-of-3 majorities, and only 0.6% are three-way splits. Pooled over 6,617 Level-4 answers with three valid verdicts.

B Released Q&A distribution

Figure 5 summarises how the released 69,691 Q&A items are distributed across levels, answer formats, and time-series context lengths. Three patterns are worth flagging.

Level sizes are uneven by design. L2 (40,226 items) is by far the largest level and L3 (2,939 items) is the smallest. The asymmetry follows directly from the underlying episode pool. Most FactoryWave episodes are anomalous (we deliberately oversample fault scenarios so that diagnostic templates have enough material), and L2 templates can be instantiated against any anomalous episode, which inflates L2’s count. Counterfactual templates (L3) by contrast require paired (baseline, fault) runs collected with the signature-kernel-MMD protocol of Section 4.2, which is far more expensive to produce per episode; the released L3 corpus is therefore the slice of items for which a high-quality CF pair exists, and is intrinsically small.

Answer-format mix tracks the reasoning level. L1 leans on numerical (69 %) and single-select MCQ (28 %) formats; L2 spreads across single-select MCQ (48 %), numerical (41 %), and multi-select (7 %), with small ranking and tensor slices (2 % each); L3 is the most format-diverse level, splitting across multi-select (42 %), numerical (21 %), ranking (20 %), and tensor (17 %), reflecting the counterfactual ranking and trajectory-prediction templates; L4 is entirely free-form because every L4 template asks for natural-language remediation or optimisation answers. Free-form appears only at L4: no L1–L3 template asks for one.

Sub-series lengths are tightly bounded. The displayed time-series window is between 32 and 77 timesteps (>64 is possible for ranking since multiple time series) for almost every released item (median ≈ 50 across all four levels), keeping the prompt within the context budget of every evaluated model.

B.1 FactoryBench-Lite

Evaluating a full panel on all 69,691 items is expensive, and the level aggregates it produces are weighted by how many items each template happens to contribute, which is a property of the episode pool rather than of the benchmark’s design (Appendix B). We therefore also release **FactoryBench-Lite**, a 3,000-item subset intended as the default entry point for cheap evaluation and for reporting that is not distorted by template mass.

Lite is not a random draw. It is balanced twice over: across templates, so no single template dominates a headline number, and *within* each template, on whatever dimension actually determines that template’s answer. The second is the substantive one. A template can be balanced overall and still be lopsided in the way that matters: the anomaly-identification templates can be even across option letters while over-representing one fault class, and the Level-4 optimisation template, which is

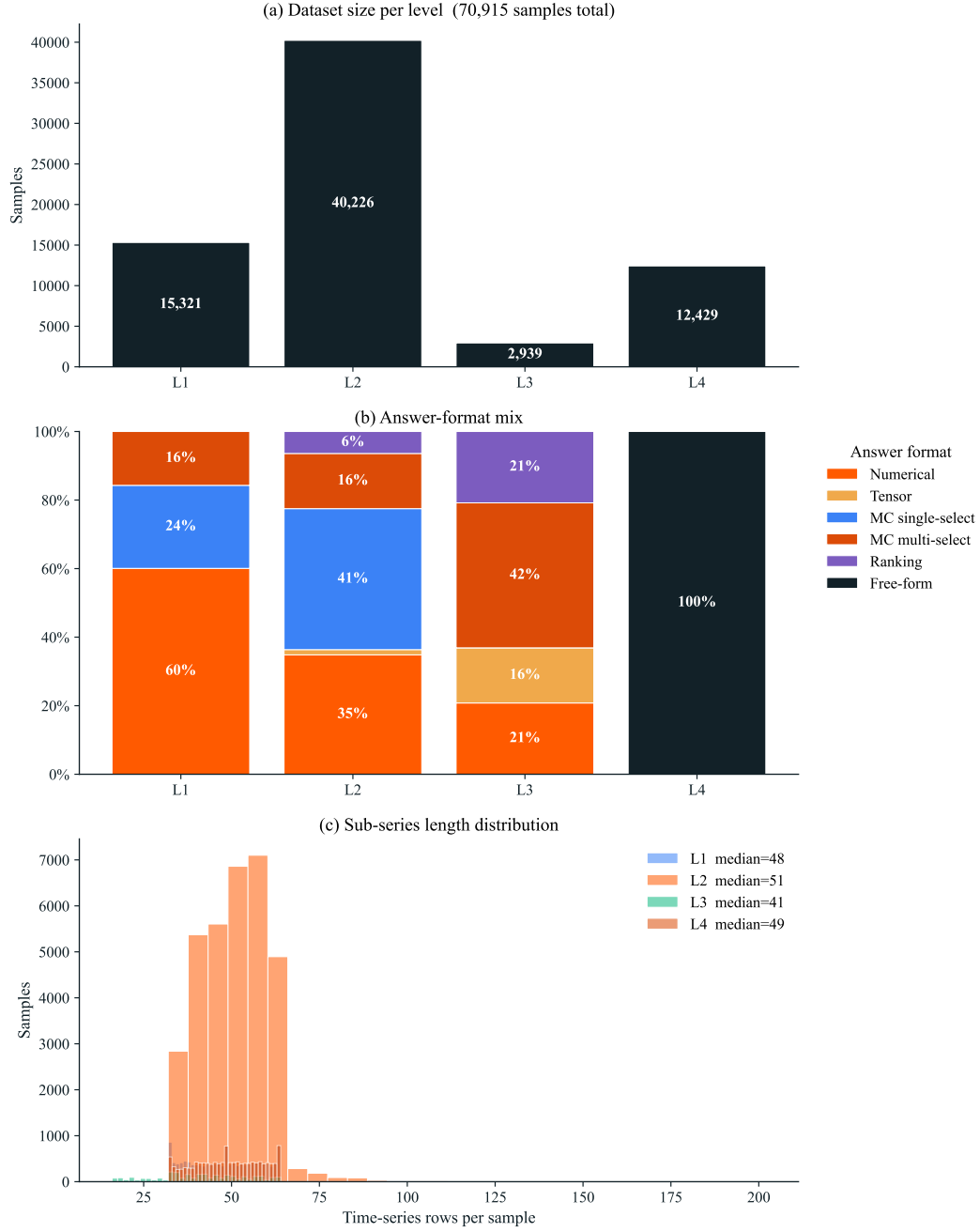


Figure 5: Distribution of the 69,691 released Q&A items. **(a)** Level sizes. **(b)** Answer-format mix per level. **(c)** Sub-series length (number of time-series rows) per item.

free-form and so has no letters to balance at all, can be sampled uniformly while still asking about payload mass far more often than about TCP frame. Each template therefore declares a stratum key – the answer string for multi-select and ranking templates, the correct class for anomaly, robot, and root-cause templates, the misconfigured parameter for optimisation, and the source episode’s (fault, task) pair for numerical templates – and selection round-robins over strata rather than sampling uniformly.

All 21 templates are represented, at 142–143 items each (572 L1, 1,430 L2, 712 L3, 286 L4). On the six multi-select templates every one of the four propositions sits at or near 50 % true, in the range [0.434, 0.503] and within 0.007 of parity on five of the six; the residual is L1.3, where the proposition in question is genuinely scarce in the source corpus. The 30 anomaly classes of L2.7 appear 4–5 times each, the 7 optimisation variants of L4.2 appear 20–21 times each, the 24 Level-4 root-cause categories appear 5–6 times each, and the robot-identification templates are even to within one item per robot. Where a stratum is genuinely scarce, round-robin takes what exists rather than over-sampling a thin class, so the achieved distribution is reported rather than assumed.

Lite is distributed alongside the full set in the same Hugging Face repository, with each item carrying its original identifier so that Lite results can be joined back to full-set results item-by-item.

B.2 Question-template inventory

Table 5 lists the number of question templates currently producing released Q&A items, broken down by answer format. Templates that are defined in the generator but do not yet emit any questions in the released set are excluded.

Figure 6 instantiates Pearl’s three causal levels (Association, Intervention, Counterfactual) on robotic time-series data and adds the Level 4 decision-making layer that FactoryBench introduces on top. Each panel pairs the formal causal primitive with a representative FactoryBench template.

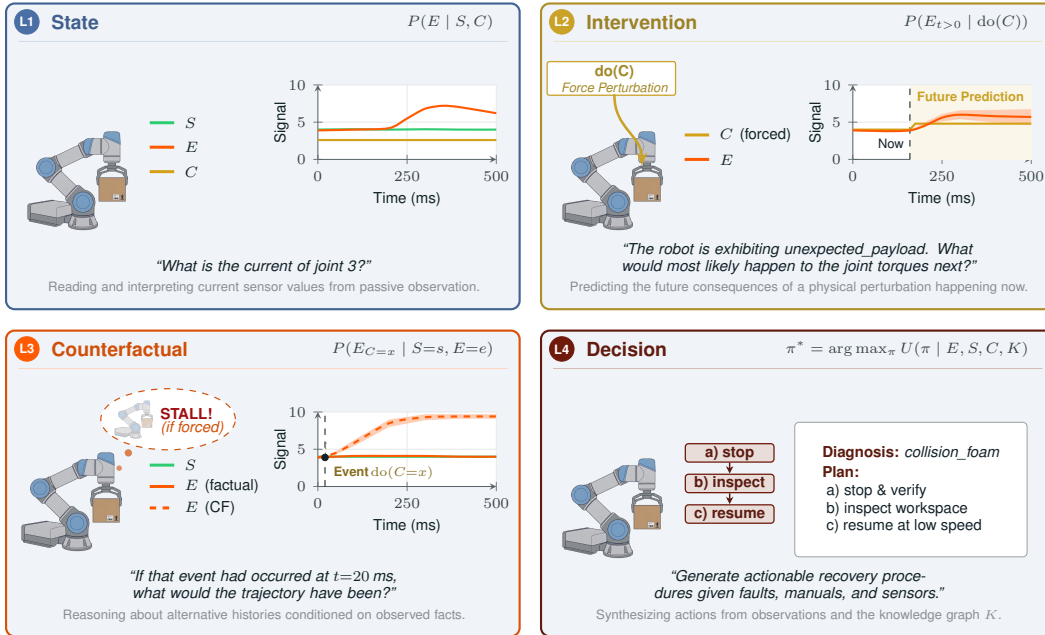


Figure 6: The four levels of machine understanding defined by FactoryBench.

Expert-authored severity ranking. Several Level-2 ranking templates (L2.1, L2.9) require ordering anomalies or signal segments by severity. Severity is not directly readable from the labels: it depends on physical impact, controller reaction, recovery cost, and downstream production consequences. The reference ordering used for these templates was authored and cross-checked by a panel of PhD-level robotics experts, who ranked each anomaly category along these axes and reviewed the resulting ranks for cross-anomaly consistency. The fixed expert ranking is shipped

Table 5: Number of active question templates per level, by answer format. Counts reflect templates that produce released Q&A items in the current dataset.

Level	MC single-select	MC multi-select	Ranking	Tensor / Numerical	Free-form	Total
1 (State)	1	1	0	2	0	4
2 (Intervention)	3	2	2	3	0	10
3 (Counterfactual)	0	2	1	2	0	5
4 (Decision)	0	0	0	0	2	2
Total	4	5	3	7	2	21

609 with the benchmark and used as the gold permutation by the deterministic positional-match scorer
610 (Appendix A).

611 B.3 Full template list

612 Table 6 lists every active template (the templates that emit released Q&A items in the current dataset).
613 Placeholders in braces ({task}, {anomaly}, {phase}, {signal}, {n}, {event}, {window_length},
614 {joint_signal}) are filled at generation time from the episode metadata or the relevance specification.
615 The trailing answer-format reminders (“Answer only with . . .”) are truncated for readability, matching
616 the convention used in Table 17.

Table 6: Full list of FactoryBench question templates by causal level. *Single-choice MCQ* requires one letter; *multi-select MCQ* requires a binary string over a fixed option set; *ranking* requires a permutation over k items; *tensor (scalar)* is a single real number accepted within a tolerance band; *tensor (6-vector)* is a JSON array of six real numbers, one per joint axis; *free-form* responses are evaluated by an LLM-as-judge against a reference answer.

ID	Answer format	Template (boilerplate truncated)
Level 1: State		
L1.1	Tensor (scalar)	The robot is performing a {task} task. We want to isolate the {phase} in the robot’s time series. Assuming a fixed window length of {window_length} timesteps, at which timestamp should the window begin?
L1.3	Multi-select MCQ	What changed between the two instances of robotic time series data?
L1.6	Single-choice MCQ	What robot does this sensor data originate from?
L1.7	Tensor (scalar)	Given the sensor stream below, what is the expected value of {signal} at T+{n}ms?
Level 2: Intervention		
L2.1	Ranking	The sensor stream below is from a robot exhibiting {anomaly}. Rank the signal segments listed in the ‘options’ field in the order you would expect them to appear as the anomaly manifests.
L2.2	Multi-select MCQ	The sensor stream below is from a robot exhibiting {anomaly}. What would most likely happen next?
L2.3	Multi-select MCQ	The sensor stream below is from a robot exhibiting {anomaly}. Select all statements that apply in the options provided.
L2.4	Tensor (scalar)	The sensor stream below is from a robot exhibiting {anomaly}. What is the expected value of {signal} at T+{n}ms?
L2.5	Tensor (6-vector)	The sensor stream below is from a robot exhibiting {anomaly}. What are the expected values of {joint_signal} at T+{n}ms?

continued on next page

Table 6 continued from previous page

ID	Answer format	Template (boilerplate truncated)
L2.6	Tensor (scalar)	Knowing that the robot suffers from {anomaly} in the given context time series, we want to isolate the {phase} phase. Assuming a fixed window length of {window_length} timesteps, at which timestamp should the window begin?
L2.7	Single-choice MCQ	Given the sensor data from a robot performing the task, determine what anomaly is present?
L2.8	Single-choice MCQ	You are provided with two sensor streams originating from robots accomplishing tasks. What differences between the two given instances of robotic time series data (if any) do you notice?
L2.9	Ranking	Rank the following robot time series segments from most to least severe anomaly.
L2.10	Single-choice MCQ	Knowing that the robot suffers from {anomaly} in the given context time series, what robot does this sensor data originate from?
Level 3: Counterfactual		
L3.1	Ranking	Given the baseline sensor stream below and the counterfactual scenario where {event}, rank the signal segments listed in the ‘options’ field in the order you would expect them to appear as the event manifests.
L3.2	Multi-select MCQ	Given the counterfactual scenario where {event}, what would most likely happen next?
L3.3	Multi-select MCQ	Given the counterfactual scenario where {event}, select all statements that would apply.
L3.4	Tensor (scalar)	In the counterfactual scenario where {event}, what would be the expected value of {signal} at T+{n}ms?
L3.5	Tensor (6-vector)	In the counterfactual scenario where {event}, what would be the expected values of {joint_signal} at T+{n}ms?
Level 4: Decision		
L4.1	Free-form	Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the most likely root cause and describe the steps you would take to fix it.
L4.2	Free-form	An engineer wants to increase the effectiveness and accuracy of this machine. Based on the sensor stream below, what operational changes or parameter adjustments could help achieve this? Do not justify your answer, simply indicate steps to take (or specify if nothing can be done).

617 **Prompt structure and knowledge-graph augmentation.** Models under evaluation receive a single
618 user message; no system role is set on the evaluated model. The message is assembled at prompt-build
619 time from five ordered blocks:

```

620 <machine_sentence>
621 <task_phase_reference>
622 <acronym_mapping>
623 <time_series>
624 Question: <question_text>
625 Here are the options:
626 <options>

```

627 The leading <machine_sentence> is filled at prompt-build time from the FactoryBench knowledge
628 graph, specifically the machine and gripper tables, keyed on the dataset that the question is grounded
629 in. This guarantees that identical question templates are grounded in the correct hardware description
630 regardless of which underlying dataset the episode comes from. For a UR3e-grounded question on
631 FactoryWave, the resulting line expands to:

632 The following sensor data comes from Universal Robots UR3e, a collaborative robot (cobot)
633 from the E-series with 3 kg payload, 6 degrees of freedom, controlled via UR PolyScope
634 (teach pendant), UrScript, typically used for light assembly tasks, automated workbench
635 scenarios. It is equipped with a 2FG14 Finger Gripper OnRobot.

636 The augmentation is conditional: identification templates such as L1.6 (“What robot does this sensor
637 data originate from?”) and L2.10 suppress the machine sentence so the model cannot read the answer
638 off the prompt header, and templates that probe gripping behavior without revealing tooling can
639 selectively suppress only the gripper clause. The acronym-mapping block decodes the time-series
640 header into long-form channel names and is included whenever the question carries one.

641 The <task_phase_reference> block defines the task’s phases, and is attached to every prompt
642 whose question or options refer to a phase, or whose context exposes the task_phase channel.
643 Without it the phase vocabulary is ambiguous in two distinct ways: the task_phase channel appears
644 in the series as a bare integer with no legend, and the question templates refer to phases in prose (“the
645 grasp of the object phase”) that does not match the canonical phase names in the knowledge graph.
646 The block resolves both by listing the phases of the episode’s task in their fixed execution order,
647 giving for each the canonical name, the wording the templates use, and a one-line description, with
648 the integer label included only when the context actually exposes task_phase. For a pick-and-place
649 episode it expands to:

650 Task: Pick and Place. Robot picks an object from a source location and places it at a target
651 location (bin). The task runs through the following phases in this fixed order, labelled by
652 the task_phase signal: 0 = Above Pick (referred to in questions as “approach to the
653 object”): EEF moves to a waypoint directly above the object. . . . 3 = Close (referred to in
654 questions as “grasp of the object”): Gripper closes around the object. . . .

655 The block is deliberately timing-free: it states what a phase *is* and where it sits in the task’s fixed
656 order, never which timesteps of the episode at hand belong to it. That distinction matters because
657 several templates (L2.6 in particular) ask the model to locate a phase boundary inside the displayed
658 window, and naming the timesteps would hand over the answer. Phase definitions are shared context;
659 phase segmentation is the thing under test.

660 Level-4 free-form responses are scored by an LLM-as-judge under a strict three-point rubric
661 ({0.0, 0.5, 1.0}) that is template-specific: troubleshooting items (template L4.1) and optimiza-
662 tion items (template L4.2) use different prompts, since the relevant axes of correctness differ (root
663 cause + remediation steps vs. problematic parameter + adjustment direction).

664 **Troubleshooting prompt (L4.1).**

665 You are scoring a model’s troubleshooting answer for a robotics
666 sensor-data benchmark.

667

668 Rubric (apply strictly, no other values allowed):

669 1.0 - the answer correctly identifies the underlying root cause AND
670 the proposed remediation steps are mostly right. The steps do
671 NOT need to match the reference exactly: minor wording
672 differences, additional sensible checks, or omitted minor
673 steps are all fine, as long as the overall remediation is on
674 the right track and would plausibly resolve the issue.

675 0.5 - the answer correctly identifies the underlying root cause but
676 the remediation is wrong, missing, or off-topic (e.g. proposes
677 an unrelated fix, contradicts the reference, or refuses to
678 give steps).

679 0.0 - the root cause is wrong or absent (model refused, identified
680 the wrong fault, or stated "no anomaly" when one was present).

681

682 The known root cause is provided to you separately. An answer counts
683 as identifying the root cause when it names the same physical
684 phenomenon - equivalent paraphrases are fine, but a different fault
685 category (e.g. saying "payload too heavy" when the root cause is

686 "TCP misconfiguration") is wrong.

687

688 Respond with ONLY a JSON object on a single line, no markdown:

689 {"score": <0.0|0.5|1.0>, "reason": "<one short sentence>"}

690

691 Question: {question}

692 Reference: {reference}

693 Known root cause:{root_cause}

694 Model answer: {prediction}

695 Optimization prompt (L4.2).

696 You are scoring a model’s optimization answer for a robotics sensor-
697 data benchmark.

698

699 Rubric (apply strictly, no other values allowed):

700 1.0 - the answer identifies the problematic parameter AND proposes
701 adjusting it in the correct direction (increase vs. decrease,
702 matching the reference). Exact magnitudes do not matter; only
703 direction matters.

704 0.5 - the answer identifies the problematic parameter but proposes
705 the wrong direction, no direction, or contradictory directions.

706 0.0 - the problematic parameter is not identified, or the model
707 recommends adjusting an unrelated parameter, or refuses to
708 answer.

709

710 The reference answer states the correct parameter and its target
711 value, from which the correct adjustment direction can be inferred
712 relative to the configured value.

713

714 Respond with ONLY a JSON object on a single line, no markdown:

715 {"score": <0.0|0.5|1.0>, "reason": "<one short sentence>"}

716

717 Question: {question}

718 Reference: {reference}

719 Model answer:{prediction}

720 The judge ensemble comprises three frontier LLMs from different vendors: **GPT-5.1**, **Claude Sonnet**
721 **4.6**, and **DeepSeek V3.2**. Each judge independently scores every Level-4 free-form response with
722 the appropriate template-specific prompt above, and the per-item score reported in our results is the
723 median of the three valid votes (snapped to the {0, 0.5, 1} grid).

724 C Episode eligibility filtering

725 Before any sub-window is drawn (Appendix D), each template restricts the pool of source episodes to
726 those that can support a well-defined ground-truth answer. The filter is template-aware and combines
727 a level-specific condition predicate with a length/coverage check.

728 **Level-specific condition predicate.** The level a template lives at fixes which episode condition the
729 template can pull from: Level 1 templates probe nominal state and therefore consume only *baseline*
730 (normal) episodes, Level 2 templates probe the immediate effect of an intervention and therefore
731 consume only *anomalous* (fault) episodes, and Level 3 templates probe counterfactual reasoning and
732 therefore consume only *counterfactual* pairs (one baseline plus one fault run, paired by the protocol
733 described in Appendix G). Level 4 templates draw from baseline and fault episodes depending on
734 the troubleshooting/optimization variant. Episodes whose condition does not match the template are
735 removed from the pool before sampling.

736 **Ground-truth feasibility.** Eligible episodes must additionally carry the labels and trajectory length
737 the template needs to derive its answer. Concretely, we require that (i) the episode contains every

738 metadata field the template instantiates (root-cause label, anomaly type, fault-injection timestep,
739 target phase, etc.), (ii) the episode is long enough that a relevance-compliant sub-window of the
740 requested length (Appendix E) fits, and (iii) for templates whose answer is computed from a future
741 segment of the trajectory (numerical and tensor forecasts at L1.7, L2.4, L2.5), the trajectory extends
742 at least the requested forecast horizon beyond the displayed window. Episodes that fail any of these
743 checks are dropped at filter time rather than at sampling time, so each template’s sample distribution
744 is over its true eligibility pool.

745 **Effect on the released set.** The filter is applied per (template, dataset, condition) cell before release,
746 so the public 70k Q&A items released on Hugging Face are exactly the items for which a verifiable
747 ground-truth answer exists.

748 **D Fault-relevance window sampling**

749 For Q&A pairs grounded in a fault episode, the time-series context shown to the model is a sub-
750 window of the full episode rather than the entire trajectory. A naive uniformly sampled sub-window
751 can miss the segment where the fault is actually visible: for a transient collision lasting tens of
752 milliseconds, a uniformly sampled one-second window has a non-negligible probability of falling
753 entirely outside the disturbed segment, producing an item whose ground-truth label is correct
754 but whose evidence is invisible to any model. To prevent this, each fault type is paired with a
755 *relevance specification* that constrains how the sub-window is drawn so that the fault signature is, by
756 construction, present in the displayed context.

757 **Four temporal footprints.** Each fault is assigned, by domain annotation, to one of four classes that
758 describe how its signature is distributed across the episode:

- 759 • *Global*: the signature is present throughout the entire episode, for instance a payload
760 misconfiguration that biases every torque reading. Sub-windows are sampled uniformly
761 under the requested length bounds.
- 762 • *Event-anchored*: the signature is a transient localized at a specific timestep recorded in
763 the episode (collisions, gripper misactivation). The sub-window is required to contain that
764 timestep; its length and start are otherwise random.
- 765 • *Phase-gated*: the signature is only diagnostic during specific task phases, for instance a
766 peg-misalignment fault that is only visible during insertion. The sub-window is required to
767 overlap one of the target phases by at least a minimum number of rows, where the relevant
768 phases depend on the task.
- 769 • *Cumulative*: the signature has a monotonically growing footprint that becomes visible
770 only after a certain phase begins, for instance progressive torque drift. The sub-window is
771 required to be at least a minimum length and, where applicable, to start no earlier than the
772 onset phase.

773 **Sampling and validation.** Given a fault episode and the requested length bounds, the sampler
774 draws a window that satisfies the corresponding constraint and records the constraint that was applied
775 alongside the question’s provenance, so downstream analyses can condition on it. A post-hoc check
776 verifies that the returned window meets the locality’s requirement; if it cannot be met (e.g., a phase-
777 gated fault whose target phase is missing from the recording), the item is either discarded or, for
778 templates where the anomaly signature is helpful but not required for correctness, the sub-window
779 falls back to uniform sampling and the item is flagged accordingly.

780 **Effect on the dataset.** Relevance-aware sampling is applied to all fault-grounded items in Levels 1,
781 2, and 4. Level 3 counterfactual pairs use a separate fixed-injection-timestep protocol described in
782 Section 4.2 and do not rely on this mechanism. In pilot generation runs without relevance constraints,
783 a non-trivial fraction of fault items had sub-windows that did not contain the fault signature, inflating
784 the apparent difficulty of Levels 1 and 2 in a way that could not be distinguished from genuine model
785 failure. The mechanism can be disabled to recover uniform sampling for ablation studies that require
786 a no-relevance baseline.

E Time-series context: features and window length

Two design choices govern what each Q&A item actually shows the model: *which* channels of the multivariate trajectory are included, and *how long* the displayed window is. Both choices are deliberately constrained.

Resampling to a uniform 10 Hz. The source datasets record at heterogeneous frequencies (83–125 Hz for FactoryWave, 100 Hz for AURSAD [1], 100 or 500 Hz for voraus-AD [2]). Before any Q&A item is built, every episode is anti-alias filtered and its *context window* is downsampled to a common nominal 10 Hz timeline. This applies to the rows the model sees in the prompt; the `timestamp_ms` column is preserved from the nearest source-rate sample and therefore is not exactly on a 100 ms grid (consecutive row offsets typically fall in [95, 105] ms depending on the native rate). Post-event windows used to compute forecast ground truth (Levels 2 and 3, templates 4 and 5) additionally retain the native sampling rate to preserve fast transients around fault onset; the answer horizon can therefore be sub-100 ms even though the visible context rows are ~100 ms apart. This context-downsampling serves three purposes: it normalizes the time axis across robots so that templates can specify window lengths in absolute seconds without conditioning on the source platform, it keeps the displayed context within the prompt budget of every evaluated model, and it removes high-frequency content that would otherwise dominate the token sequence with information not needed for the reasoning levels we evaluate. Ten Hertz is admittedly low by industrial-sensor standards: at higher sampling rates one can resolve fast transients (millisecond-scale collisions, valve switches) that are smoothed away here. We accept this trade-off because the fault catalogue evaluated in this paper is deliberately simplified to atomic, well-isolated mechanisms, and we observe empirically that 10 Hz is sufficient to expose every fault signature in our catalogue at a level that PhD-level annotators can verify on the displayed context. Future versions of the benchmark targeting compound or sub-second faults would need to revisit this choice.

Per-template feature pre-selection. The raw signal schema is large: FactoryWave exposes more than 120 channels per timestep (joint positions, velocities, accelerations, currents, target torques, contact forces, TCP pose, gripper state, and so on). Including every channel verbatim would produce contexts that are both prohibitively long and dominated by signals irrelevant to the question being asked. We therefore equip every question template with a hand-curated subset of *important features*: 30 channels per template for Levels 1–3, and roughly 20 for Level 4 (whose templates focus on the diagnostic and remediation signals that matter for troubleshooting). The selections are authored by PhD-level robotics experts on the team and target the channels most informative for the specific reasoning skill the template probes; for instance, an anomaly-detection template emphasizes torques and contact forces, while a phase-isolation template foregrounds joint positions and velocities. The full per-template feature lists are released alongside the benchmark so that every Q&A item can be reproduced bit-for-bit from the underlying episode.

Window length. Across all four levels, each Q&A item shows a sub-window of between 32 and 64 timesteps drawn from the 10 Hz-resampled trajectory, corresponding to roughly 3.2–6.4 s of robot motion. The lower bound is set so that even the shortest displayed window contains enough samples to expose the dynamics the templates probe (transient onsets, phase boundaries, force spikes); the upper bound keeps the prompt within the context limits of all evaluated models, including the smaller open-weight ones. Within these bounds, the exact length and start position are randomized at generation time, with the start position constrained by the relevance specification of Appendix D when the item is fault-grounded. These bounds apply per displayed window rather than per item, so templates that show more than one window carry a proportionally longer total context: comparison templates present two sub-windows side by side, and ranking templates add four candidate segments on top of the context window.

Why these choices matter for interpretability. Fixing the resampling rate, the feature subset, and the window-length range across templates means that performance differences between models on a given template cannot be attributed to one model receiving a longer, faster, or richer context than another, only to its ability to interpret the same content. The randomization within bounds prevents models from exploiting positional or length cues to recognize templates by their surface form: two items instantiated from the same template are typically presented with different lengths, different

start positions, and – once relevance constraints and per-template feature subsets are honoured – with the fault evidence falling at different relative positions in the displayed window.

F Validation of solvability

This appendix collects the diagnostic checks we ran to convince ourselves that FactoryBench items are answerable from the released time-series context rather than from question wording alone.

F.1 Human expert baseline

The strongest evidence that an item is solvable from the released telemetry is a human solving it. We therefore had a robotics expert answer a 60-item sample spanning Levels 1–3 under open-tool, no-LLM conditions, and scored the answers with the benchmark’s own grader.

Protocol. *Open tools.* The annotator was free to use any analysis instrumentation, and built a small Python toolkit for the task: series parsing, windowed condition search and threshold crossings, linear extrapolation, per-channel signal statistics, forward-kinematics consistency checks, and nominal-versus-actual absolute-difference comparison. The reference we want is what a competent engineer with normal instrumentation can extract from the telemetry, not what a person can do reading serialized numbers unaided. *No LLM assistance.* The annotator did not use a language model to produce or check any answer, so the baseline is independent of the systems under evaluation. *Identical items, identical scoring.* The annotator worked from the same items and prompts the model panel saw, and the answers were scored by the benchmark’s own grader: the same acceptance bounds for scalar and vector answers, the same exact-match rule for single-select and ranking, and the same per-position partial credit on multi-select. No separate human rubric was introduced, and the human receives no credit the panel would not receive. *Duration.* The entire process took a total of two work days of question answering and tool design by the expert.

Composition. 13 items at Level 1, 37 at Level 2, and 10 at Level 3, drawn across 14 distinct templates and covering all five answer formats. We plan on extending the baseline to address this current imbalance.

Table 7: Human expert baseline against the model panel, raw scores on the identical 60-item sample. The expert reaches 0.917 overall (49/60 items exactly correct).

	Expert	Claude Sonnet 4.6	Mistral Large 3	GPT-5.1	Qwen3 235B	DeepSeek V3.2
Level 1	0.962	0.712	0.500	0.635	0.519	0.481
Level 2	0.980	0.662	0.444	0.392	0.385	0.351
Level 3	0.625	0.325	0.550	0.500	0.575	0.675

The expert clears 0.96 at both Level 1 and Level 2 while the best model sits at 0.68 and the rest between 0.39 and 0.46. This settles the solvability question directly: the items are answerable from the released signals, so the low model scores reflect model limitations rather than broken or ambiguous items. It also shows that human experts are capable of a much deeper comprehension of machine state, whether healthy or anomalous, even though that comprehension is arrived at through a time-consuming process of measurement and computation (usually causing costly downtime) rather than intuitive signal comprehension.

The margin is not uniform across templates. It is narrowest where the answer can be read off a single channel with a short calculation, and widest on the templates that require committing to a specific root cause or a specific machine from the signal.

Level 3 is materially harder for the expert too. At Level 3 the expert reaches 0.625 with only 3 of 10 items exactly correct, and is not the top scorer on that level. This is generally explained by how undercovered and difficult the problem of counterfactual prediction is in time series analysis, leading to a lack of specific tools: the toolkit that carries the expert to near-ceiling on Levels 1–2 has no counterpart for reasoning about an intervention that was never executed. We read this as a

property of the level rather than of the items, and it tempers the interpretation of the Level 3 column in Figure 3: the human ceiling there is much closer to the panel than at Levels 1–2.

Extension. We are extending the baseline to at least 100 Q&A pairs with a minimum of 25 per level, which means adding the Level 4 decision-making questions and broadening the Level 1 and Level 3 samples. We commit to reporting the full per-level results in an updated appendix for the camera-ready version, and to releasing the baseline itself: for every item, the expert’s answer, its score, and the method used to reach it.

F.2 Distribution-shift analysis

We further confirm solvability by inspecting how the sensor data distribution shifts when a fault is present. Figure 8 illustrates this by comparing multiple healthy KUKA episodes (blue) against multiple faulty episodes with an arm-disturbance fault (orange) drawn from FactoryWave (Figure 7 shows the physical setup of the fault for visual intuition), and reveals a clear distributional shift between the two groups on the channels the affected templates read from. The shift is more pronounced on some joints than on others, as the angle at which the disturbance is applied loads them unequally.

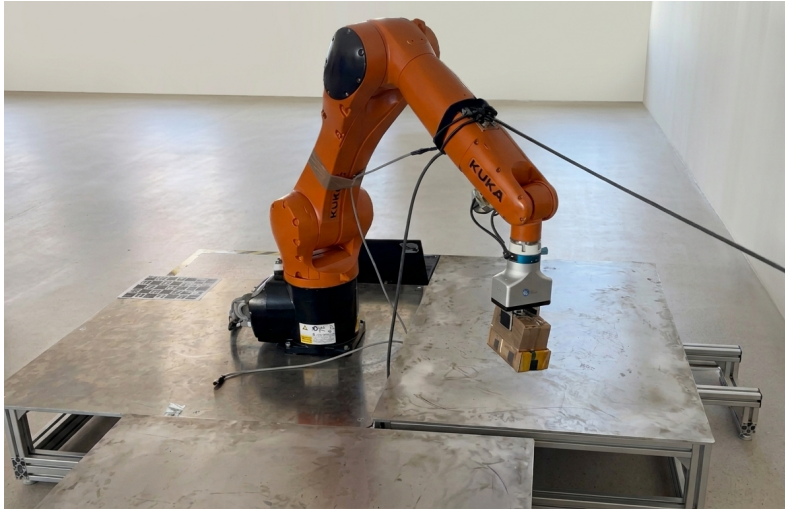


Figure 7: The arm-disturbance fault as physically applied to the KUKA in FactoryWave.

G Details of the FactoryWave dataset

FactoryWave was collected on two one-armed platforms: the **Universal Robots UR3** (cobot; OnRobot Screwdriver and two-finger gripper variants) and the **KUKA KR10** (industrial arm). Each episode corresponds to one complete task cycle; episodes are recorded under three conditions (normal, fault, and counterfactual), described below. Because state interpretation rather than policy quality is the object of study, motion is generated by a deterministic scripted controller. Every sample carries extensive labeling metadata, and the full native-rate sensor stream is logged without online feature selection. Table 8 gives the breakdown of the 8,983 episodes across robots, tasks, and conditions.

G.1 Tasks and phase structure

Three industrial tasks are represented in the dataset: Pick-and-Place, Screwing, and Peg-in-Hole. All three are executed on both robots, using matched scripted controllers and task layouts so that cross-embodiment comparisons hold the task fixed. We structure each task as a deterministic sequence of phases (Tables 9, 10, 11); the phase index is written as a discrete label at every timestep, providing an interpretable segmentation that downstream models can condition on or that can be used to localize questions to a specific stage of the cycle. Within this fixed phase skeleton, we randomize as much of the episode as the task permits so that a question template that targets a given phase does not accidentally learn a fixed pose, timing, or trajectory. Pick and place locations are sampled uniformly

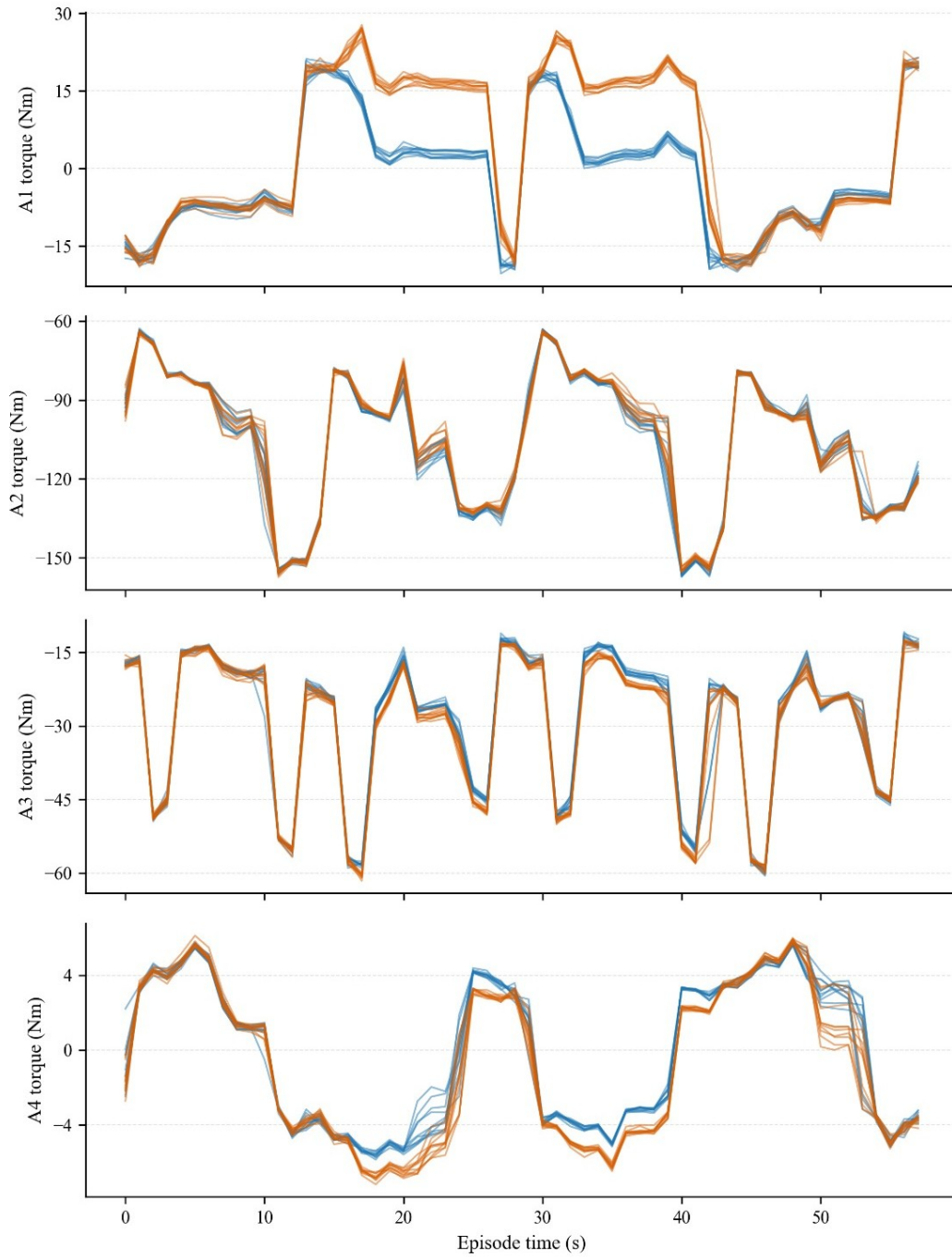


Figure 8: Sensor data distribution across multiple healthy KUKA episodes (blue) versus multiple faulty episodes with an arm-disturbance fault (orange) in FactoryWave. The visible shift between the two groups confirms that the affected templates are still solvable from the released signal; the magnitude of the shift varies across joints because the disturbance angle loads them unequally.

Table 8: Distribution of FactoryWave episodes by robot, task, and condition. Counterfactual entries count CF groups (one baseline plus one retained fault run per group), as each one represents data of one studied scenario.

Robot	Task	Normal	Fault	CF groups	Total
UR3	Pick-and-Place	1053	3238	109	4400
UR3	Screwing	230	850	40	1120
UR3	Peg-in-Hole	322	1200	100	1622
KUKA KR10	Pick-and-Place	578	1221	42	1841
Total		2183	6509	291	8983

from predefined workspace regions; phase-specific joint velocities, accelerations, and end-effector (EEF) rotations are drawn from bounded per-phase ranges; and approach angles to contact events are perturbed within the tolerances of the controller. This forces the Q&A pairs generated from FactoryWave to reflect phase-level reasoning rather than memorisation of a single scripted trajectory, which in turn makes them generalizable to a wide variety of trajectories and variation found in real recordings.

Table 9: Task phases for Pick-and-Place. EEF refers to End-Effector

Phase index	Phase name	Description
0	above_pick	EEF moves to a waypoint directly above the object
1	descend	EEF lowers toward the object
2	settle	Brief hold to let physics settle before grasping
3	close	Gripper closes around the object
4	lift	EEF lifts the grasped object off the surface
5	move_xy	Lateral transfer toward the bin
6	lower	EEF lowers into the bin
7	open	Gripper opens, object released
8	retract	EEF moves away from the bin
9	return	EEF returns to home configuration

Table 10: Task phases for Screwing.

Phase index	Phase name	Description
0	approach	EEF moves to a pre-contact waypoint above the fastener
1	descend	EEF lowers to engage the fastener
2	screw	Continuous downward force + wrist rotation to thread
3	disengage	Gripper opens / tool lifts off
4	retract	EEF returns to a safe height
5	descend	EEF lowers to engage the fastener (loosening pass)
6	loosen	Loosen the screw
7	engage	Gripper closes on the retrieved screw
8	retract	EEF returns to home position

G.2 Episode metadata

Alongside the sensor stream, every episode carries a fixed set of metadata fields describing the robot, task, condition, payload, and end-effector configuration (Table 12). These fields are constant within an episode and are used both as filters during question generation (e.g. to restrict a template to a specific payload regime) and as provenance in the released Q&A pairs. This includes fault-specific fields (such as injection timestamp or physical offset).

G.3 Fault taxonomy

FactoryWave includes 27 distinct fault types (Table 13). Each fault is paired with a plain-language description of the underlying physical or control-level mechanism and the injection procedure used to

Table 11: Task phases for Peg-in-Hole.

Phase index	Phase name	Description
0	above_hole	EEF moves above the hole and aligns
1	insert	EEF pushes peg downward into the hole
2	disengage	Gripper opens
3	above_hole	EEF rises back to the waypoint above the hole
4	retract	EEF returns to home position
5	above_object	EEF moves above the object and aligns with it
6	engage	Gripper closes
7	above_object	EEF rises back to the waypoint above the object
8	retract	EEF returns to home position

Table 12: Default per-episode metadata fields.

Field	Description
episode_id	Unique identifier for the episode
robot_model	Robot model (UR3, KUKA KR10)
task	Task (pick-and-place, screwing, peg-in-hole)
condition	Episode condition (normal, fault, counterfactual)
fault_id	Fault index for fault episodes; null otherwise
weight_of_box	Mass of the transported object in kg (pick-and-place only)
shape_of_box	Shape/dimensions of the transported object (pick-and-place only)
position_of_box	Initial XYZ position of the object on the table, where measurable
gripper_model	Gripper model and type (vacuum, two-finger, screwdriver)
payload_mass_configured	Payload mass configured at the controller (kg)
payload_cog_configured	Payload CoG offset configured at the controller (x, y, z in mm)
tcp_offset_configured	TCP position offset configured at the controller (x, y, z in mm)
tcp_orientation_configured	TCP orientation configured at the controller (rx, ry, rz in radians)

926 induce it during data collection. The last three columns indicate which tasks include the fault in the
 927 collected dataset; faults that apply to all three tasks (e.g. collisions with a shared workspace object,
 928 misconfigurations of the shared controller) are recorded under each applicable task with task-specific
 929 signal imprints.

Table 13: Fault catalogue. Columns *PP*, *Scr*, *PiH* indicate whether the fault is included in the Pick-and-Place, Screwing, and Peg-in-Hole recordings.

Fault		Explanation	Injection	PP	Scr	PiH
Damaged thread	screw	Screw thread physically damaged, preventing proper engagement during tightening.	Pre-damaged the screw thread with sandpaper.	×	✓	×
Missing screw		Tightening is attempted with no screw present.	Removed the screw from screwdriver before the cycle.	×	✓	×
Damaged plate thread		The threaded hole in the plate is damaged, preventing engagement.	Pre-damaged the plate hole with a metal screwdriver.	×	✓	×
Loosening phase		A counterclockwise loosening rotation replaces tightening; a normal phase with a distinct signal. It is counted here as an anomaly to match the fault schema of the AURSAD dataset [1].	Reversed the rotation direction in the controller program to execute a counterclockwise loosening pass in place of tightening.	×	✓	×

Table 13 (cont.)

Fault	Explanation	Injection	PP	Scr	PiH
Gripper activation failure	The vacuum gripper fails to activate and never picks up the box.	Disabled the gripper activation command in the robot program so the pick cycle completes without a grasp.	✓	×	×
Gripper release during motion	The gripper releases the payload mid-trajectory, causing an abrupt payload loss.	Triggered a programmatic gripper-release command at a scripted timestep mid-trajectory.	✓	×	×
Additional axis payload	A dead weight attached to one link increases inertia and gravity loading on all joints.	Bolted calibrated weights of varying mass to one robot link.	✓	✓	×
Collision with foam object	Contact with a soft foam block produces a brief TCP force spike without a protective stop.	Placed a foam cube directly in the programmed TCP trajectory.	✓	✓	✓
Unexpected payload weight	The transported box has a weight that deviates from the nominal payload configuration.	Swapped the nominal box for a known heavier or lighter one. This is done in a counterfactual context, ie. switching weights and asking about alternate weight scenario.	✓	×	×
Invalid gripping position	Timing error: the gripper closes after the arm has begun lifting, gripping the object off-center.	Inserted a brief delay in the controller program so gripper closure lags the lift motion.	✓	×	×
Unstable mounting platform	Base instability introduces low-frequency vibrations into the arm.	Placed 8 layers of towels under the base plate.	✓	×	×
Joint position limit violation	A joint moves outside its configured soft or hard position limits, triggering a safety stop.	Random waypoint slightly beyond the configured soft joint limit.	✓	×	×
TCP frame misconfiguration	TCP frame or mounting orientation misconfigured at the controller; gravity torques deviate.	Entered an incorrect TCP offset or mounting angle in the controller installation settings.	✓	✓	✓
Payload weight misconfiguration	Payload mass misconfigured while a tool or workpiece is physically attached.	Left the configured payload weight at multiple wrong mass configurations while a task-dependent gripper remained mounted.	✓	✓	×
External arm disturbance	A continuous external force pulls or pushes the TCP during motion.	Anchored an elastic rope between the bench frame and the tool flange.	✓	✓	✓
Mild payload CoG misconfiguration	Payload CoG specified at an incorrect offset from the tool flange; gravity compensation is wrong.	Entered a CoG offset in the payload configuration that differs from the physical tool CoG.	✓	✓	✓
Collision with hanging cable	A loose cable drags along a robot link or the tool flange during motion.	Suspended a loose cable across the trajectory at arm height.	✓	✓	✓

Table 13 (cont.)

Fault	Explanation	Injection	PP	Scr	PiH
Collision with cardboard object	A cardboard carton in the trajectory provides moderate resistance; may trigger a protective stop.	Placed a free-standing or braced cardboard box in the programmed trajectory.	✓	✓	✓
Collision with rigid object	A rigid plastic block in the TCP path triggers an immediate protective stop.	Clamped a rigid plastic block in the programmed TCP path. Unlike the other collisions, this fault consistently causes safety stops.	✓	✓	✓
Peg insertion misalignment	Peg approaches the hole at an angular or lateral offset and jams against the rim.	Added a recorded lateral offset at the insertion approach waypoint.	×	×	✓
Hole obstruction	Foreign object or debris in the hole prevents full insertion.	Placed a small object (metal screw) inside the hole before insertion.	×	×	✓
Incorrect insertion depth	Insertion terminates at an incorrect Z depth due to a misconfigured waypoint.	Shifted the insertion endpoint waypoint Z by recorded offset from nominal.	×	×	✓
Peg surface contamination	Contamination or roughness on the peg raises insertion friction and produces stick-slip.	Applied dry chalk powder to the peg surface.	×	×	✓
Fixture displacement	Insertion fixture shifted laterally, breaking the match between programmed approach and actual hole.	Physically shifted the hole fixture by recorded offset without updating the robot program.	×	×	✓
Self-collision	Arm configuration causes a link to contact the robot body or mounting fixture, triggering a protective stop.	Random waypoint that forces a self-colliding configuration or offset the mounting fixture into the trajectory.	✓	×	×
Missing box	Box absent from the pick position; the gripper closes on empty air.	Removed the box from the pick position before the episode started.	✓	×	×
Missing peg	Arm descends into the hole empty-handed; insertion produces no contact force.	Removed the peg from the gripper before the episode started.	×	×	✓

930 G.4 Counterfactual episodes

931 Counterfactual pairs are constructed for the subset of faults in Table 13 whose injection moment can be
932 controlled at a specific timestep rather than being present throughout the episode. For each such fault,
933 the initial conditions of the physical setup (object identity and pose, payload configuration, controller
934 settings) are held as constant as the platform allows, one baseline execution is recorded without any
935 injection, and several fault executions are then recorded under the same initial conditions with the
936 fault applied at a shared sampled timestep. Because the physical system is not perfectly reproducible,
937 the pre-injection sensor trajectories of the baseline and the fault candidates diverge slightly; we retain,
938 as the counterfactual partner of the baseline, the fault candidate whose pre-injection window is closest
939 to the baseline under the signature kernel MMD, and discard the others. This yields an approximate
940 counterfactual pair in which the early history is as matched as the real platform permits, while the
941 post-injection divergence isolates the effect of the intervention. The approximation error introduced
942 by this procedure, relative to the exact counterfactuals available in simulation, motivates the sim2real
943 analysis in Section J.

G.5 Signal schema

The UR3 is recorded at 125 Hz, yielding 136 per-sample signals grouped by semantic role (Table 14): controller *setpoints*, actual *effort / feedback* readings from joints and the TCP, *context / health* signals covering joint temperatures, voltages, and internal controller state, a small block of controller *status / mode* indicators, and digital/analog *I/O* lines. A handful of episode-level provenance fields (episode identifier, robot type, task, fault label, payload mass, recording date) are appended per episode but are constant within an episode.

Table 14: UR3 signal groups recorded at 125 Hz (excluding episode-level provenance columns).

Group	# signals	Contents
Setpoint (intent)	42	Target joint position, velocity, acceleration, current, torque; target TCP pose and speed.
Effort / feedback	48	Actual joint position, velocity, current, current-as-torque, joint control output; actual TCP pose, speed, and force/torque.
Context / health	33	Joint temperatures, joint voltages, joint modes, tool accelerometer, raw F/T wrench, main/robot voltage and current, momentum, speed scaling, execution time.
Status / mode	7	Timestamp, robot mode, robot status, safety mode, safety status bits, runtime state, configured payload mass.
I/O	6	Digital inputs/outputs, two analog inputs, two analog outputs.
Total	136	

The KUKA KR10 is controlled through the KRC4 controller. We stream 53 per-sample signals at 83 Hz, grouped by semantic role in Table 15 using the same partition as the UR3 schema. A handful of signals that the UR3 exposes natively are unavailable on KSS 8.3, specifically joint velocities, commanded TCP pose, joint voltage, and joint-level control outputs; TCP force/torque is also absent because no force sensor is fitted. To recover tool-side feedback, we mount a 9-DoF inertial measurement unit on the gripper and log its triaxial linear acceleration (acc_x/y/z), angular rate (gyro_x/y/z), and orientation (angle_x/y/z) at the controller’s 83 Hz cadence.

Table 15: KUKA KR10 signal groups recorded at 83 Hz via RSI/KRL (excluding episode-level provenance columns).

Group	# signals	Contents
Setpoint (intent)	6	Commanded joint positions (degrees).
Effort / feedback	24	Actual joint positions (degrees), actual TCP pose (mm / degrees), per-axis motor current (% of max), and per-axis motor torque (Nm).
Context / health	11	Per-axis motor temperature (Kelvin), Cartesian acceleration on X, Y, Z and absolute magnitude (m/s^2), and speed override (%).
Tool IMU	9	Gripper-mounted 9-DoF IMU: triaxial linear acceleration acc_x/y/z (m/s^2), angular rate gyro_x/y/z (rad/s), and orientation angle_x/y/z (degrees).
Status / mode	1	Controller process state (FREE / ACTIVE / STOP / END).
I/O	2	Digital inputs bitmask and digital outputs bitmask.
Total	53	

H Time-series foundation model baseline (Chronos-Bolt)

To check whether the LLM panel is the right comparison set on a benchmark whose motivation references time-series models, we evaluate CHRONOS-BOLT [25] (~ 200 M parameters), a pretrained probabilistic time-series forecaster, as a non-LLM baseline in the same zero-shot setting (no fine-tuning, prompting, or context-window engineering on either side).

Template applicability. A pure forecaster has neither a language interface nor a notion of options, ranking, or counterfactual conditioning. Of the 21 active question templates in FactoryBench, only three reduce to a well-posed forecasting problem: L1.7 (scalar forecast), L2.4 (scalar forecast under a known anomaly), and L2.5 (six-joint tensor forecast under a known anomaly). The remaining 18 templates (MCQ classification, ranking, counterfactual conditioning, free-form root-cause and protocol writing) are structurally outside the scope of any TSFM that lacks a language head. **This 14 % template coverage is itself a finding:** any TSFM, regardless of size or pretraining quality, can address only the forecasting slice of FactoryBench, and the other 86 % of the benchmark measures capabilities that no current TSFM provides.

Setup. For each forecasting item we form a univariate context per target channel, request the median quantile at the requested horizon, and score under the same chance-corrected piecewise-margin rule used in the main paper (Appendix A; $E = 1/4$). For the tensor template (L2.5) we run six independent per-joint forecasts and average the per-channel scores. Chronos-Bolt has no multivariate or text-conditioned mode, so cross-channel correlations and the anomaly named in the question are unavailable to it.

Results. On the three forecasting templates (Table 16, $n = 469$), Chronos-Bolt achieves chance-corrected accuracy of 51.5%, 46.9%, and 49.7% on L1.7, L2.4, and L2.5 respectively (signed correction). As a reference point, the strongest LLM in the panel reaches +46.8% on the L1 level aggregate (Section 5.1) and +25.8% on the L2 aggregate; a per-template comparison at that granularity is left to follow-up work.

Table 16: CHRONOS-BOLT zero-shot performance on the three forecasting templates of FactoryBench ($n = 469$). Confidence intervals are 95 % percentile intervals from 1000 bootstrap resamples. Chance-corrected (CC) scores apply the signed transform $(s - 1/4)/(1 - 1/4)$.

Template	n	Raw mean	Raw 95% CI	CC mean	CC 95% CI
L1.7 (state, scalar)	334	0.636	[0.588, 0.681]	0.515	[0.451, 0.575]
L2.4 (intervention, scalar)	74	0.601	[0.493, 0.710]	0.469	[0.324, 0.614]
L2.5 (intervention, tensor[6])	61	0.623	[0.537, 0.708]	0.497	[0.383, 0.611]

Score distribution and signal coverage. Per-item scores are strongly bimodal: items tend to land at 1.0 or 0.0, with a smaller mass at the half-credit band, characteristic of a margin-thresholded scorer. Aggregating items by target-signal family shows roughly uniform capability across the four families (positions, speeds, torques/efforts, contact forces) within bootstrap noise, indicating the baseline does not selectively succeed on smooth signals and fail on noisy ones. Score declines monotonically with forecast horizon, as expected for a probabilistic forecaster.

Fairness of comparison. The comparison is apples-to-apples on item identity, scoring rule, chance correction, and zero-shot setting. It is *not* apples-to-apples on (i) input representation: LLMs see acronymized text-encoded values, alongside the question, robot description, and option set, while Chronos-Bolt sees raw univariate float arrays only; and (ii) question awareness: on L2.4 and L2.5 the LLM is told which anomaly is present, while Chronos-Bolt cannot use that information. A multivariate or text-conditioned TSFM (e.g. Moirai-MoE, Time-MoE) might exploit these signals; we do not, to keep the baseline architecturally minimal. We therefore report Chronos-Bolt numbers as template-specific rather than as level aggregates.

Interpretation. Two messages emerge. **First, on the templates where forecasting is the right tool, a 200 M-parameter pretrained forecaster is competitive with frontier LLMs**, consistent with the main paper’s observation that LLMs struggle with precise numerical state extraction at L1. The LLM panel is not the only, nor the most viable model class on the forecasting slice. This opens the door to tool-assisted LLM agents that delegate forecasting subtasks to a specialised TSFM and reserve the LLM for the linguistic, classification, and decision-making templates where it has the structural advantage. **Second, the remaining 18 of 21 FactoryBench templates (classification, ranking, counterfactual reasoning, free-form troubleshooting and optimization) are structurally outside any current TSFM’s scope.** Together these reinforce the paper’s central claim: the gap

1006 between current models and operational machine understanding is not closed by solely scaling LLMs
 1007 or deploying purpose-built forecasters.

1008 I Example question-answer pairs and question templates

1009 Table 17 presents seventeen representative Q&A pairs spanning every causal level and every answer
 1010 format the generator produces (single-select MCQ, multi-select MCQ, tensor, ranking, free-form).
 1011 For readability we omit the underlying sensor context (tens to hundreds of time-series rows), truncate
 1012 verbose boilerplate such as “Answer only with ...”, and abbreviate long option strings.

Table 17: Representative Q&A pairs from FactoryBench spanning all four causal levels and every answer format.

Level	Answer format	Prompt, options, and reference answer
L1	Tensor	Q: The robot is performing a screwing task. We want to isolate the screwing phase in the robot’s time series. Assuming a fixed window length of 10 timesteps, at which timestamp should the window begin? A: 14 (accepted within ± 3)
L1	Single-select MCQ	Q: What changed between the two instances of robotic time series data? Options: a. different anomalous states (exactly one anomalous, or both but distinct) b. different tasks c. different robots d. same task, different phases A: a
L1	Single-select MCQ	Q: What robot does this sensor data originate from? Options: a. Agile Robots Yu 5 Industrial b. KUKA KR 10 R1100-2 c. Universal Robots UR3 A: c
L1	Tensor	Q: Given the sensor stream below, what is the expected value of the position of joint 2 at T+606 ms? A: 102.7777
L2	Tensor	Q: The sensor stream below is from a robot exhibiting a collision with a cardboard object. What is the expected value of the velocity of joint 3 at T+68 ms? A: -0.272044 (accepted within ± 2.42)
L2	Tensor	Q: The sensor stream below is from a robot exhibiting a collision with a cardboard object. What are the expected values of joint positions at T+506 ms? A: [17.884, -115.897, 121.049, -95.595, -90.258, 293.860]
L2	Tensor	Q: Knowing that the robot suffers from a hole obstruction, we want to isolate the “rise back above the object” phase. Assuming a fixed window length of 16 timesteps, at which timestamp should the window begin? A: 202 (accepted within ± 3)
L2	Single-select MCQ	Q: Given the sensor data, determine what anomaly is present. Options: a. sustained command/measurement deviation b. no anomaly c. measured joint speeds drop abruptly with a TCP force spike (typical collision signature) d. isolated unrealistic force-sensor spike inconsistent with motion A: c
L3	Ranking	Q: Given the baseline stream and the counterfactual scenario where a collision foam object occurs at timestep 7366 ms, rank the four labeled signal segments in the order they would appear as the event manifests. A: abdc

continued on next page

Table 17 continued from previous page

Level	Answer format	Prompt, options, and reference answer
L3	Multi-select MCQ	Q: Given the counterfactual scenario where a collision hanging cable occurs at timestep 8363 ms, select all statements that would apply. Options: a. motor current rises $\geq 33\%$ while speed stays $\leq 10\%$ of baseline (stall-like) b. contact-force spike $\geq 36\%$ above baseline c. tracking-error rise $\geq 34\%$ above pre-event mean d. at least one joint speed drops sharply ($\geq 31\%$) A: FFFT
L3	Tensor	Q: In the counterfactual scenario where a collision foam object occurs at timestep 7659 ms, what would be the expected value of the position of joint 2 at T+100 ms? A: 72.144939 (accepted within ± 2.77)
L4	Free form	Q: Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the most likely root cause and describe the steps you would take to fix it. A: No anomalous behavior detected; the machine is operating normally; no remediation is required.
L4	Free form	Q: Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the most likely root cause and describe the steps you would take to fix it. A: a. Inspect the workspace and remove any soft obstructions from the programmed trajectory. b. If a protective stop occurred, acknowledge it in the robot controller and verify arm posture before resuming. c. Reduce speed if repeated false-positive collision detections occur.
L4	Free form	Q: Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the most likely root cause and describe the steps you would take to fix it. A: a. Stop the robot and update the payload configuration to include the additional axis weight. b. Set the correct CoG offset for the modified arm in the installation settings. c. Reduce motion accelerations and speeds to account for increased effective inertia. d. Run a slow-speed test cycle to verify dynamic loads stay within rated limits before resuming.
L4	Free form	Q: An engineer wants to increase the effectiveness and accuracy of this machine. Based on the sensor stream below, what operational changes or parameter adjustments could help achieve this? A: The payload mass is set to 0.0 kg but the actual payload weighs 1.5 kg. Update the payload mass in the installation settings to 1.5 kg.
L4	Free form	Q: An engineer wants to increase the effectiveness and accuracy of this machine. Based on the sensor stream below, what operational changes or parameter adjustments could help achieve this? A: The TCP offset is misconfigured at [0, 0.3, 55] instead of the correct [0, 0, 55]. Update the TCP position offset in the installation settings to [0, 0, 55].
L4	Free form	Q: An engineer wants to increase the effectiveness and accuracy of this machine. Based on the sensor stream below, what operational changes or parameter adjustments could help achieve this? A: The payload center of gravity is set to [25, 0, 20] but the correct value is [0, 0, 20]. Update the CoG offset in the installation settings to [0, 0, 20].

1013 J Simulation setup and sim-to-real gap analysis

1014 We run sim2real rollouts in Isaac Sim using the built-in UR3 asset, with phase-consistent waypoint
1015 replay of the recorded `target_joint_*` signals. The simulated task is UR3 pick-and-place, follow-
1016 ing the 10-phase structure of Table 9. Payload settings are matched episode-wise to the FactoryWave
1017 chronology.

1018 Each real episode is exported, converted to a per-episode simulation config, and replayed in Isaac.
1019 Simulated outputs preserve the FactoryWave field schema (`joint_*`, `tcp_*`, `task_phase`, etc.)
1020 and are paired with their real counterpart by episode ID. Gap metrics are computed phase-wise on
1021 aligned trajectories: joint RMSE (deg), TCP position RMSE (mm), TCP rotation error (mrad), and
1022 Wasserstein-1 on `joint_current_*` (A).

1023 Across 1,155 paired episodes, pooled per-episode metrics are: joint RMSE mean/median =
1024 3.65/2.83 deg, TCP position RMSE = 13.16/13.26 mm, and W1 effort mean = 0.83/0.81 A. TCP
1025 rotation error remains broad (median 2605 mrad), indicating orientation mismatch is the least stable
1026 component. Translational kinematics and effort-proxy alignment are substantially more consistent
1027 than rotational alignment.

1028 K Representation probing: does the model encode what it cannot report?

1029 FactoryBench, as presented in the main text, is a purely *behavioural* evaluation: a model is prompted
1030 with telemetry and graded on its final answer. A wrong answer therefore conflates two qualitatively
1031 different failure modes: (a) the relevant machine concept is *absent* from the model’s internal repre-
1032 sentation, or (b) the concept is *present* in the representation but the model cannot surface it through
1033 its output. Distinguishing them matters for interpreting the Level-4 collapse of Section 5.3, and for
1034 localising where progress must come from (better decoding versus better pretraining). We separate
1035 the two with a targeted linear-probing study [49] on an open-weight panel member, in the spirit of the
1036 geometric analyses of motion transformers in *Words in Motion* [50]. All numbers below reproduce
1037 from the released probing code.

1038 **Setup.** We probe **Qwen3-4B** (frozen, no fine-tuning). Each item is passed through the model once
1039 on the *exact* prompt the zero-shot panel saw, and we cache the residual-stream activation at every
1040 layer (0=embedding through 36) at two read-out positions: the last prompt token, and the mean over
1041 the time-series token span. All activations are computed with causal attention, verified by a startup
1042 self-test that checks that layer- k predictions are invariant to removing later tokens.

1043 **Probes and controls.** For each (concept, layer, read-out) we fit an L_2 logistic-regression probe on
1044 the top-256 principal components of the standardised activations, with the standardiser and PCA fit
1045 *inside* each cross-validation fold so no validation point shapes its own projection. The regularisation
1046 strength is chosen by *episode-grouped* 3-fold cross-validation, and the reported layer is the one
1047 that maximises this CV score (never the test accuracy, to avoid a layer-selection peek). A one-
1048 hidden-layer MLP is a non-linear reference. Four controls guard the read-out: (i) **selectivity** [51],
1049 the probe accuracy minus that of an identical probe trained on *shuffled* labels, which stays near zero
1050 unless the probe reads a real concept rather than memorising; (ii) a **raw-signal baseline**, the same
1051 probe fit on per-channel signal statistics (mean/std/min/max/last, with the timestamp stripped so no
1052 positional information leaks), which measures what is decodable from the numbers with *no model*;
1053 (iii) a **random-init control**, the same probe pipeline on a same-architecture network with *untrained*
1054 weights, which measures how much of the decodability is generic random-feature structure of the
1055 input rather than something the model learned in pretraining; and (iv) **episode-disjoint** train/test
1056 splits with bootstrap 95% confidence intervals that resample whole *episodes* (not items, which are
1057 correlated within an episode). Headline probe accuracies are the mean over three random seeds (split,
1058 PCA, and C -selection). We probe two machine concepts. Neither concept is a benchmark label in its
1059 own right: both are derived from the fault identifier recorded with each episode. *Fault type* collapses
1060 that identifier to the six most frequent injected mechanisms, chosen so the classes carry comparable
1061 numbers of items and a six-way probe is not dominated by one mechanism; the long tail of rarer
1062 faults is excluded rather than merged into a catch-all, which would make the sixth class a mixture and
1063 the accuracy uninterpretable. *Fault presence* is the binary reduction of the same identifier, healthy
1064 against any injected fault.

1065 **Behavioural read-out a .** To place each probe against the model’s own competence we ask the
1066 same frozen model the concept directly, on the same held-out probe items: fault presence as a yes/no
1067 question, and fault type as an A–F multiple choice over exactly the probe’s six classes, with the
1068 option-to-letter assignment *randomised per item* so a positional answer bias cannot masquerade as
1069 knowledge (scored against the recorded fault identifier). We let the model reason (chain-of-thought,
1070 up to 1024 tokens for the six-way question), take its committed answer when it gives one, and for
1071 items where it reasons without committing we read its choice *parse-free* from the logits over the
1072 option tokens after a “therefore the answer is” cue, so every item is scored and no inference is
1073 discarded. Probe accuracy p and behavioural accuracy a are computed on the identical test items,
1074 giving a matched read: p asks whether the concept is decodable, a whether the model uses it.

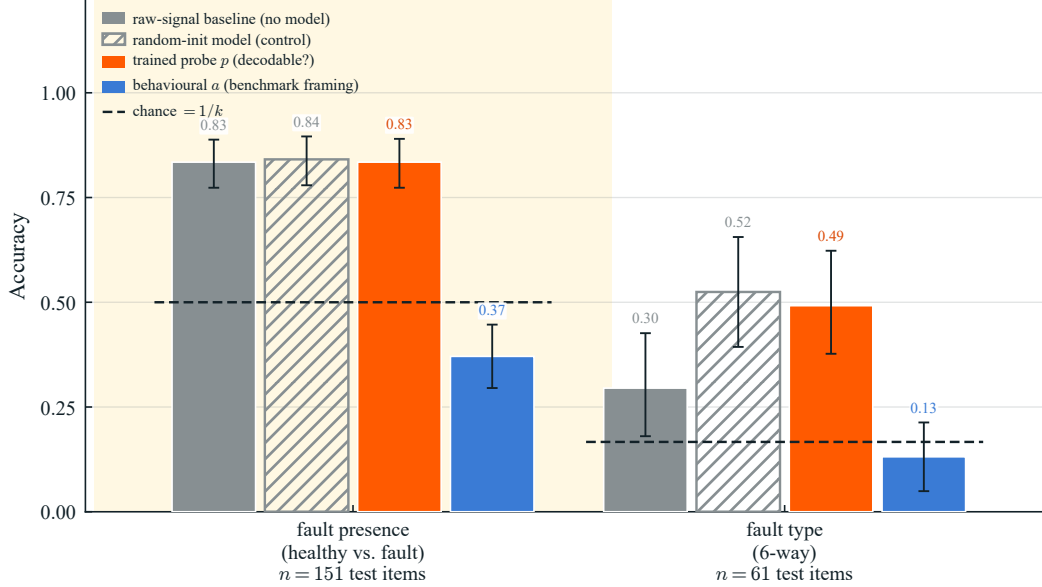


Figure 9: **Decodable is not the same as learned, and neither is the same as reported.** For fault presence and fault type we compare the trained linear probe’s accuracy p (is the concept decodable from Qwen3-4B’s frozen residual stream?) with the model’s own behavioural accuracy a on the same question, alongside a raw-signal baseline (no model), a random-init control (same architecture, untrained weights), and chance. Fault presence is decodable at $p=0.83$, but the model never approaches it behaviourally: $a=0.37$ under the benchmark’s anomaly wording, where it replies “anomalous: yes” to every episode, far below 0.83. Raw signal statistics (0.83) and even an untrained network (0.84) decode it just as well, so this availability is surface, not learned. Fault type is only weakly decodable ($p=0.49$, CV-selected) and, crucially, *not above the random-init control* (0.52), which in fact edges it out; the model answers at $a=0.13$. Chance is $1/k$ (uniform random guessing): 0.5 for the binary presence task, $1/6$ for the six-way type task. Each probe is read from the CV-selected best layer for the last-token read-out, layer 2 for fault presence and layer 1 for fault type; the layer is chosen by episode-grouped cross-validation on the training folds and never on the test split, so the reported accuracy is not a layer-selection peek. Error bars are episode-level bootstrap 95% CIs and are shown for *every* condition, including the raw-signal and random-init baselines, so no bar is read as a point estimate against an interval-bearing one; accuracies are on the held-out test split. The random-init control is run at the same sample size, seed count and split as the trained probes (600 items per concept, three seeds, 151/61 test items), so the intervals are directly comparable. They are wide relative to the gaps: on fault presence the trained probe, the raw-signal baseline and the random-init control overlap almost entirely, and on fault type the control’s estimate (0.52) sits slightly *above* the trained probe’s (0.49), with each inside the other’s interval. This is why we read the fault-type result as no learned gain rather than a small one.

1075 **Fault presence is decoded but not reliably reported.** Fault presence is linearly decodable at
1076 $p=0.83$ (three-seed mean 0.86 ± 0.02 ; mean-pool 0.86; selectivity 0.34, so this is not probe mem-
1077 orisation). The model’s behavioural read-out, by contrast, is unreliable. On the benchmark’s own

anomaly wording it answers “yes” to *every* episode and scores $a=0.37$, below the 0.5 chance of random guessing, because only 38% of episodes are faulty, so an always-“anomaly” policy scores at that base rate (Figure 9, left). The model therefore applies a miscalibrated policy biased toward predicting a fault, and does not approach the 0.83 that is linearly available. The gap is moreover entirely at the read-out: the raw-signal baseline (0.83) and the random-init network (0.84) both decode fault presence just as well, so it is an “easy” property already carried by signal statistics and even by untrained features, to which the pretrained model adds no representational value.

Fault type is neither learned nor reported. Fault *type* is only weakly decodable: $p=0.45\pm0.07$ (three-seed mean; the CV-selected layer is shallow, e.g. layer 1), above the raw-signal baseline (0.30) but *not* above the random-init control (0.52 at its CV-selected layer, 0.45 ± 0.06 across three seeds against the trained probe’s 0.45 ± 0.07). All three clear the $1/6\approx0.17$ chance of the six-way task, so the signal does carry some fault-type structure, but the trained probe extracts no more of it than an untrained network. A same-architecture network with untrained weights decodes fault type as well as the pretrained one, and its per-layer curve tracks the trained probe at every depth (Figure 10); the trained model’s marginal contribution over an untrained one is within noise. This decodability therefore reflects generic random-feature structure of the input rather than a representation the model learned in pretraining. Behaviourally the model reaches only $a=0.13$, below the $1/6$ chance of the six-way task. Given a chain-of-thought budget it does commit an answer on 84% of items, and its errors now spread across five of the six classes: the earlier apparent “collapse onto two classes” was an artefact of a fixed option order, which we remove by randomising the A–F assignment per item. For fault type, then, we find neither a learned representation nor a usable read-out.

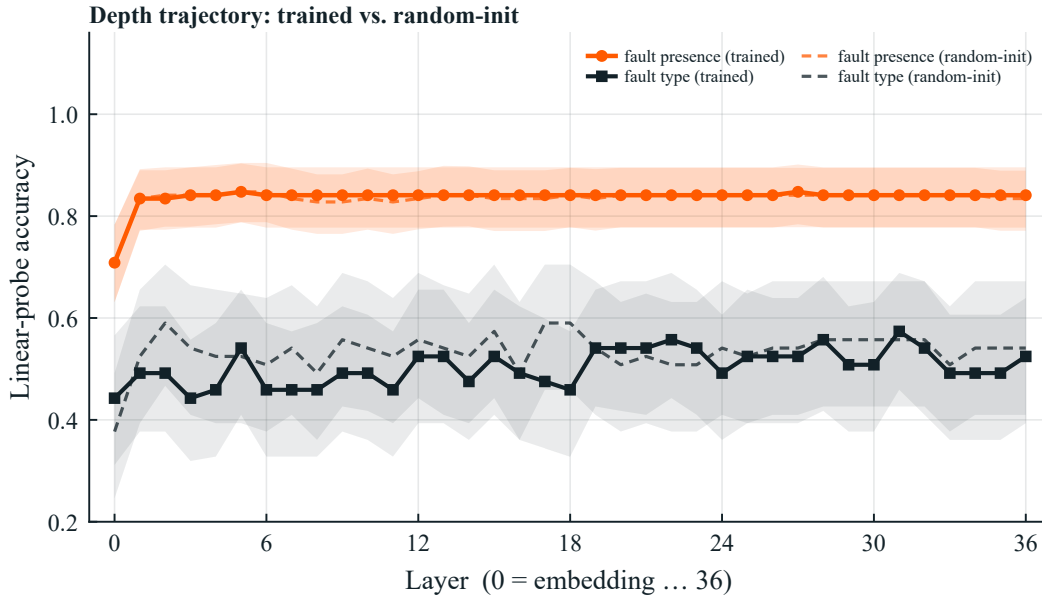


Figure 10: **An untrained network decodes fault type as well as the trained one.** Per-layer linear-probe accuracy (last-token read-out), trained (solid) vs. random-init control (dashed). Fault presence is decodable already at the embedding (0.71) and saturates immediately, but the random-init curve matches it. Fault type’s trained curve tracks its random-init curve at *every* depth, showing no learned gain, and the CV-selected best is shallow. Behavioural read-out accuracies for both concepts are reported in Figure 9. Episode-level bootstrap 95% CIs are shaded for both the trained probes and the random-init controls, which are distinguished by their dashed centre lines; for fault type the two bands overlap at every depth. Trained probes and control are run at the same sample size, seed count and split, so the bands are directly comparable.

Takeaway. The two concepts sit at different points on a representation–read-out spectrum. Fault *presence* is a *read-out* failure: it is trivially decodable ($p=0.83$, matched by raw signal statistics and by an untrained network), yet the model answers below chance (0.37), applying an anomaly-predicting policy it cannot calibrate. Fault *type* is weaker still: the pretrained model shows no

1103 decodable representation beyond a random-init baseline (0.45 ± 0.07 vs. 0.45 ± 0.06 across three
 1104 seeds) and cannot report it ($a=0.13$): a representational gap, not merely a decoding one. Part of the
 1105 Level-4 collapse of Section 5.3 is therefore a genuine read-out/calibration failure where the concept
 1106 is present, and part reflects the absence of a learned representation where it is not. Near-term progress
 1107 will need both: better decoding and calibration for the surface concepts, and better representation
 1108 learning, from pretraining or fine-tuning on a frozen backbone, for the computed ones.

1109 **Scope.** The study probes a single open-weight model (the only panel member exposing activations
 1110 and within computational reach) and two concepts with dense ground truth. Two cautions on
 1111 interpretation. First, a linear probe measures a *lower bound* on decodability under a specific read-out:
 1112 a near-chance probe bounds what a linear read-out recovers, not the information content of the
 1113 signal, so our fault-type result is “the model does not linearly represent this beyond random features,”
 1114 not “the information is absent” (the random-init control already recovers 0.44, so the signal is not
 1115 empty). Second, the probe is supervised on frozen features whereas the model answers zero-shot,
 1116 an asymmetry intrinsic to probing, where p measures latent *availability* and a measures zero-shot
 1117 *usability*. Full per-layer numbers for both read-outs, all four controls, the multi-seed spreads, and the
 1118 exact reproduction command are released with the benchmark code.

1119 L Causal schema (SCE): full details

1120 This appendix gives the long-form definition of the Setpoint–Context–Effort + Feedback (SCE)
 1121 schema used to organize every time-series channel in FactoryBench, together with the residual-based
 1122 fault definition that the schema enables. The motivation is summarized in Section 3.2; here we cover
 1123 the per-group channel breakdown, the residual formalism, and the per-robot calibration that makes
 1124 faults computable rather than imputed.

1125 The schema groups every recorded channel into one of three causal roles. **Setpoint** captures the
 1126 controller’s commanded target: per-axis position, velocity, and acceleration setpoints emitted by the
 1127 trajectory generator. **Context** captures physical and configured conditions affecting behavior, split
 1128 into *static* context (episode-level metadata such as payload mass, payload center of gravity, TCP
 1129 offset, gripper model, and box / peg geometry) and *dynamic* context (per-sample channels such as
 1130 joint temperatures, supply voltages, controller mode, and safety mode). **Effort + Feedback** captures
 1131 the machine’s measured response: motor currents and torques (effort), measured joint positions and
 1132 TCP pose (feedback), tool accelerometer, and acoustic emission where available. The exact per-robot
 1133 channel mapping appears in the signal-group tables in this appendix (UR3 and KUKA KR10).

This split enables a single operational definition of a fault that applies across machines and tasks.
 Under healthy operation, the measured Effort + Feedback response $\mathbf{y}_t$ is by definition a function of
 the Setpoint command $\mathbf{S}$ and the Context $\mathbf{C}_t$:

$$\mathbf{y}_t = f(\mathbf{S}, \mathbf{C}_t),$$

1134 where f represents the nominal plant dynamics (inverse-dynamics-plus-controller transfer) calibrated
 1135 per robot. Faults manifest as structured residuals from this expected baseline: $\mathbf{y}_t - f(\mathbf{S}, \mathbf{C}_t)$ forms
 1136 a clear, fault-specific spatiotemporal pattern. Because every episode supplies the paired $(\mathbf{S}, \mathbf{C}_t, \mathbf{y}_t)$
 1137 streams, this residual is computable directly from the data, giving ground truth that does not have to
 1138 be hand-imputed. Figure 11 summarises the causal relationships between the three groups.

1139 M License and availability

1140 FactoryBench and FactoryWave are freely available for academic and commercial use under a two-
 1141 track licence: the **generator source code, evaluation scripts, and tooling** (this repository) are
 1142 released under the **MIT License**, and the **data and benchmark artefacts** (FactoryWave episodes,
 1143 question templates, paraphrase banks, LLM-as-judge prompts, and the full Q&A dataset) are re-
 1144 leased under **Creative Commons Attribution 4.0 (CC BY 4.0)**, with attribution to be given to the
 1145 FactoryBench authors. Both artefact tracks are distributed via the public Hugging Face repository
 1146 FactoryBench/FactoryBench. To guard against test-set contamination in future model releases,
 1147 we hold back a private 15% subset of the Q&A pairs at the *episode* level (so all questions grounded in
 1148 any held-out episode stay private across all four levels) and keep its raw episode data, gold answers,
 1149 and rubric annotations off the public release; this private slice is used internally for contamination

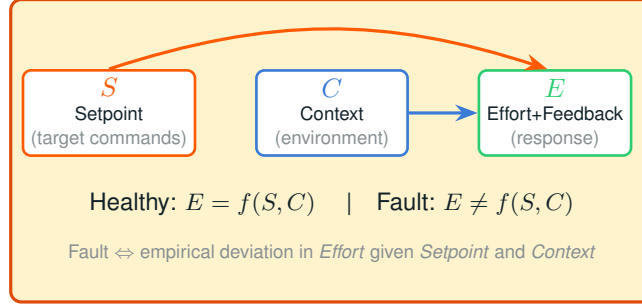


Figure 11: Physical causal schema. Setpoint commands S and contextual variables C jointly determine the measured Effort + Feedback response E under healthy operation. Faults manifest as residuals $E - f(S, C)$ from this nominal mapping.

1150 checks and may also be used to score future evaluatees on request. The remaining 85% is fully public
 1151 and is released as a single undivided pool, alongside the balanced 3,000-item FactoryBench-Lite
 1152 subset of Appendix B.1.

1153 We provide a versioned release track (e.g., v1.0, v1.1) so that reported numbers always refer
 1154 to a fixed benchmark state; corrections to labels or templates are published as minor-version
 1155 updates with a public changelog, and the exact version used in any evaluation is stamped into
 1156 every result file. **All counts, tables, and results reported in this paper correspond to re-**
 1157 **lease FactoryBench/FactoryBench:v1.0.0, the paper-submission snapshot** (69,691 released
 1158 Q&A items grounded in 8,983 FactoryWave episodes). The live main branch of the dataset
 1159 repository may contain additional episodes and Q&A items added after the submission snap-
 1160 shot; those additions are versioned in the changelog and do not retroactively affect the num-
 1161 bers here. Bug reports and label-correction requests are tracked on the public repository at
 1162 <https://anonymous.4open.science/r/FactoryBench/README.md>.